MidcurveNN

Computing Midcurve using Neural Networks

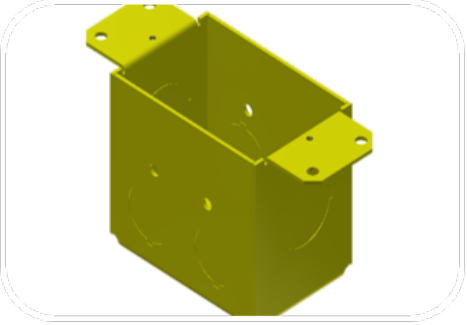
Yogesh Haribhau Kulkarni

YHK

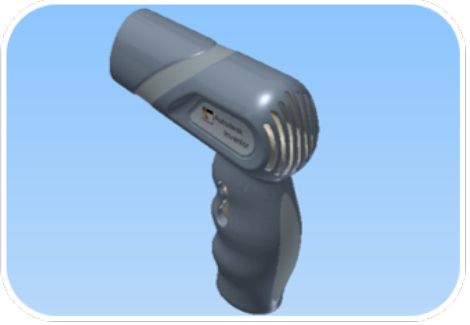
Introduction To Midcurve



Aerospace



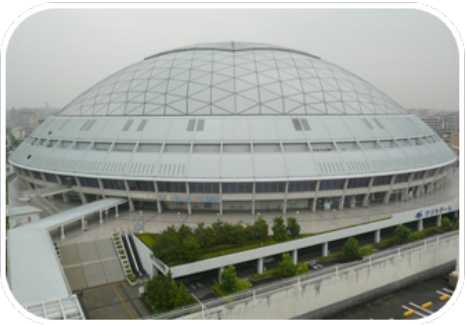
Machinery



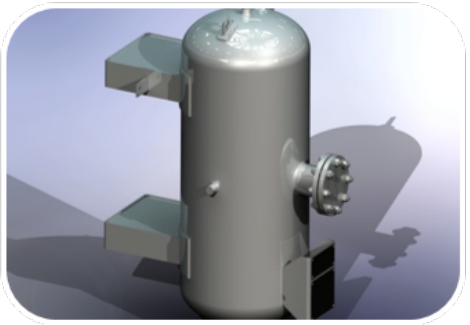
Consumer Products



Energy



Construction

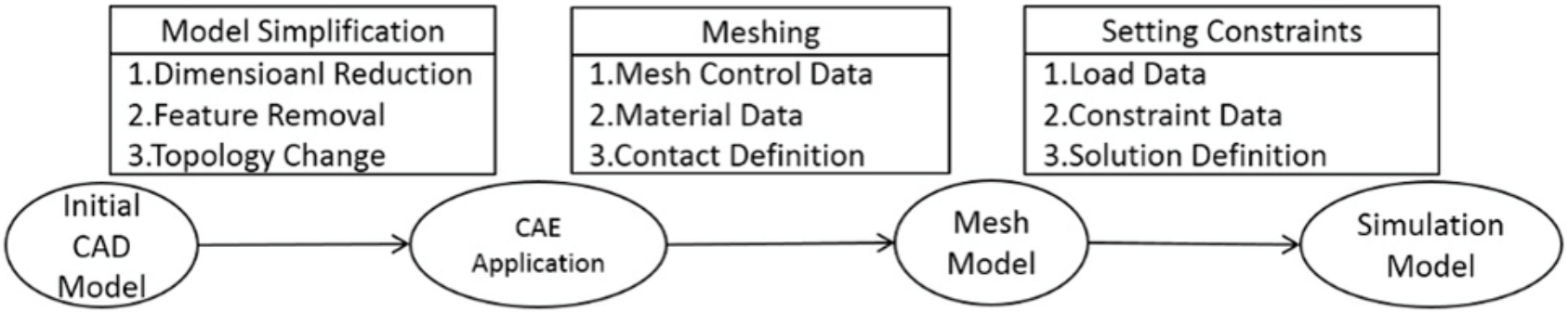


Industrial equipment

YHK

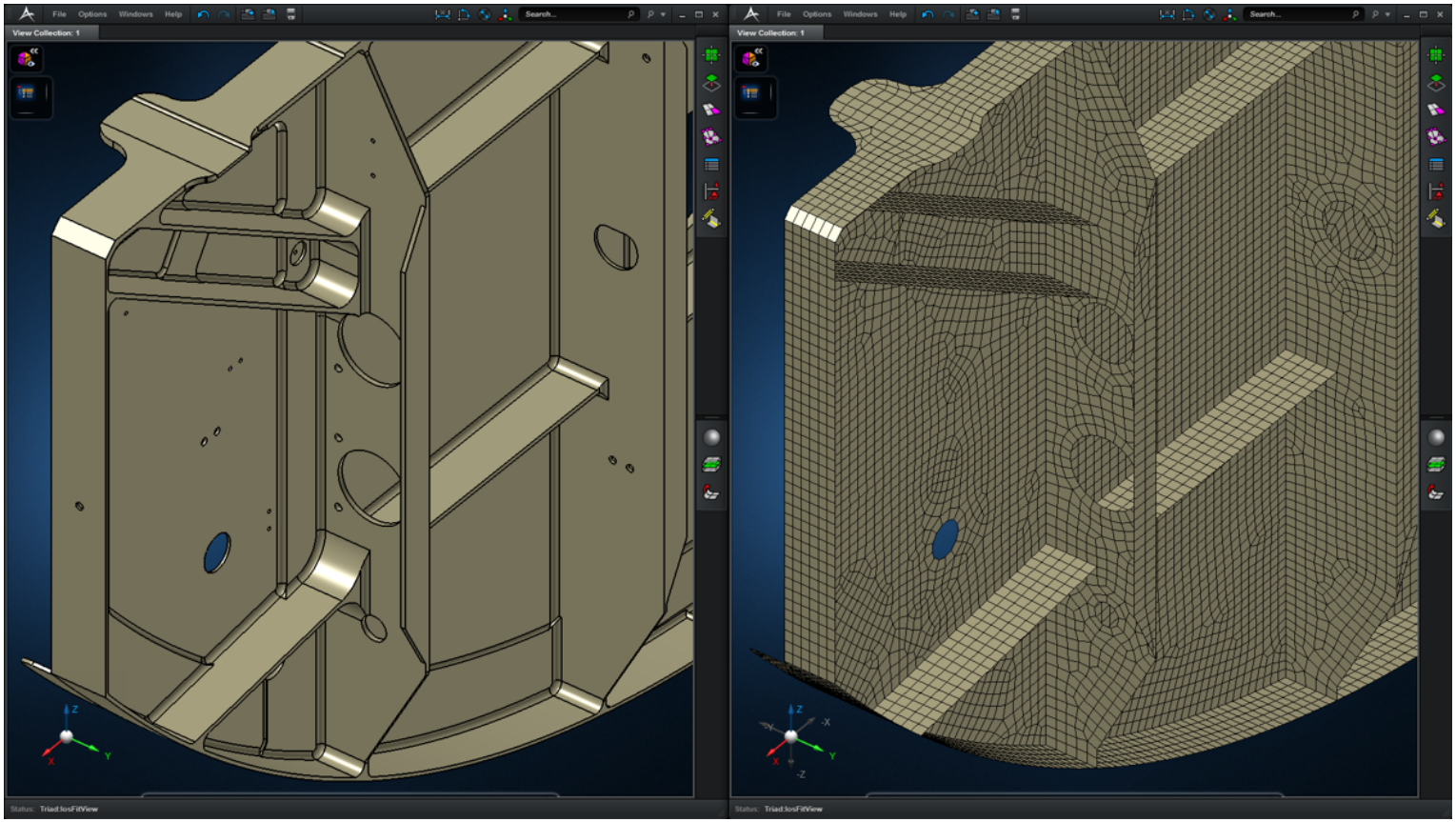
Can we use shapes directly?

- CAD : Designing Shapes
- CAE : Engineering Analysis
- CAD→CAE: Simplification for quicker results.



YHK

CAD-CAE



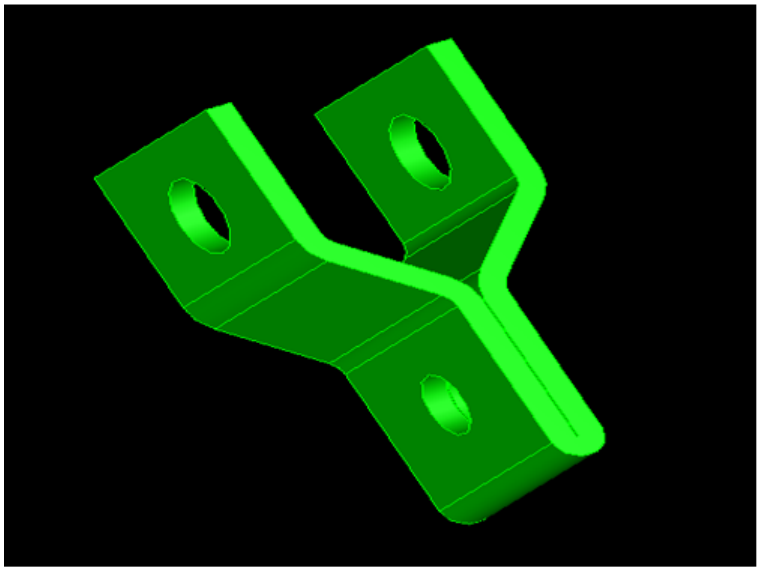
For Shapes like Sheet Metal ...

	Solid mesh	Shell+Solid mesh	Difference (%)
Element number	344,330	143,063	-58%
Node Number	694,516	75,941	-89%
Total Degrees of freedom	2,083,548	455,646	-78%
Maximum Von. Mises Stress	**418.4 MPa**	**430 MPa**	+3%
Meshing + Solving time	Out of memory	22 mins	N/A (4G RAM)
Meshing + Solving time	**30 mins**	**17 mins**	-43% (12G RAM)

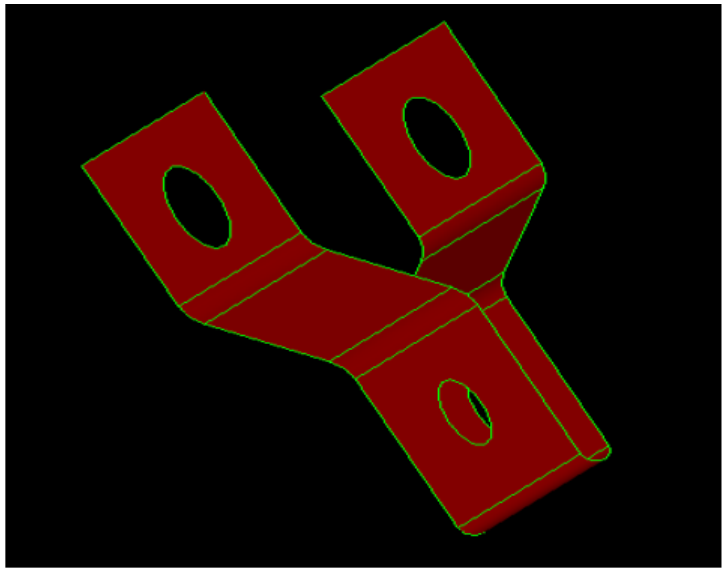
Half the computation time, but similar accuracy

YHK

Midsurface is?



Input: Solid



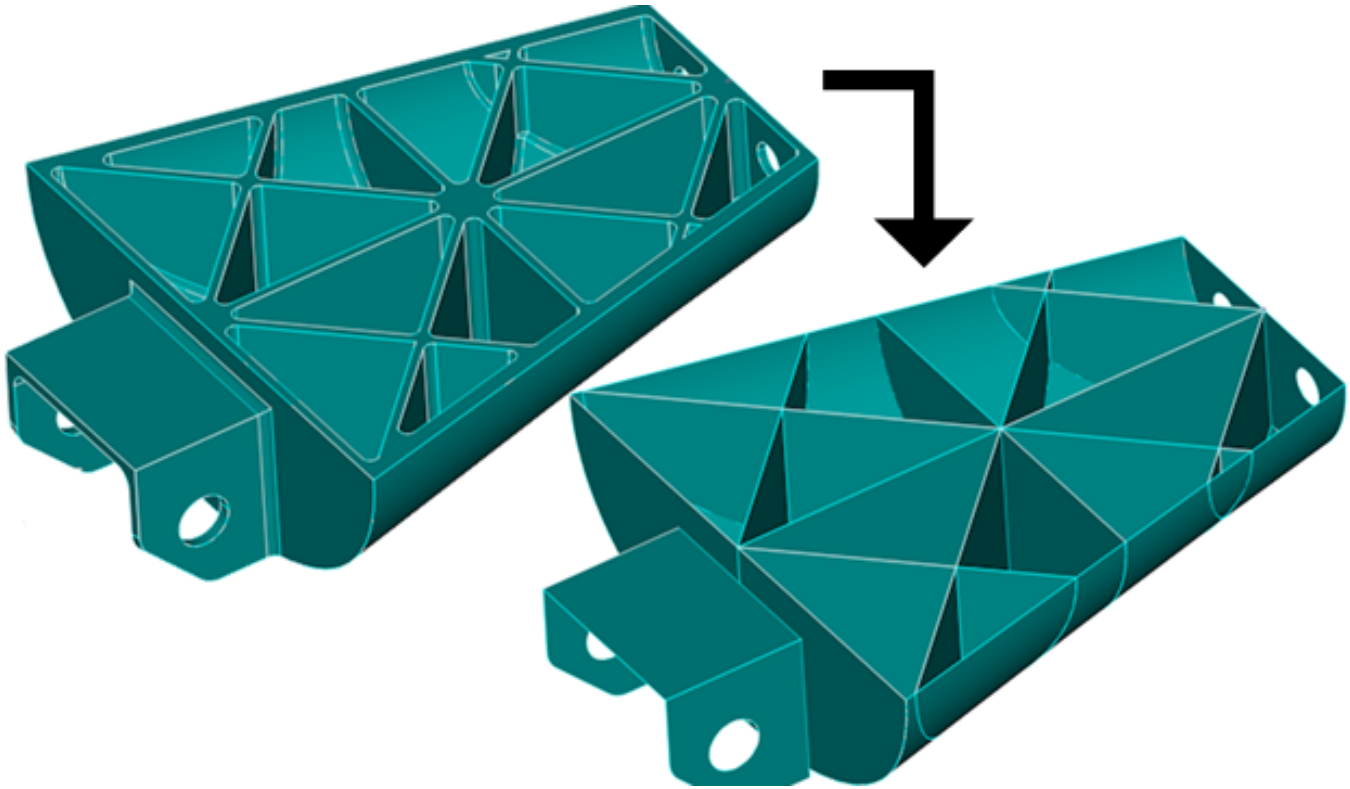
Output: Midsurface

- Widely used for CAE of Thin-Walled parts
- Computation is challenging and still unsolved

YHK

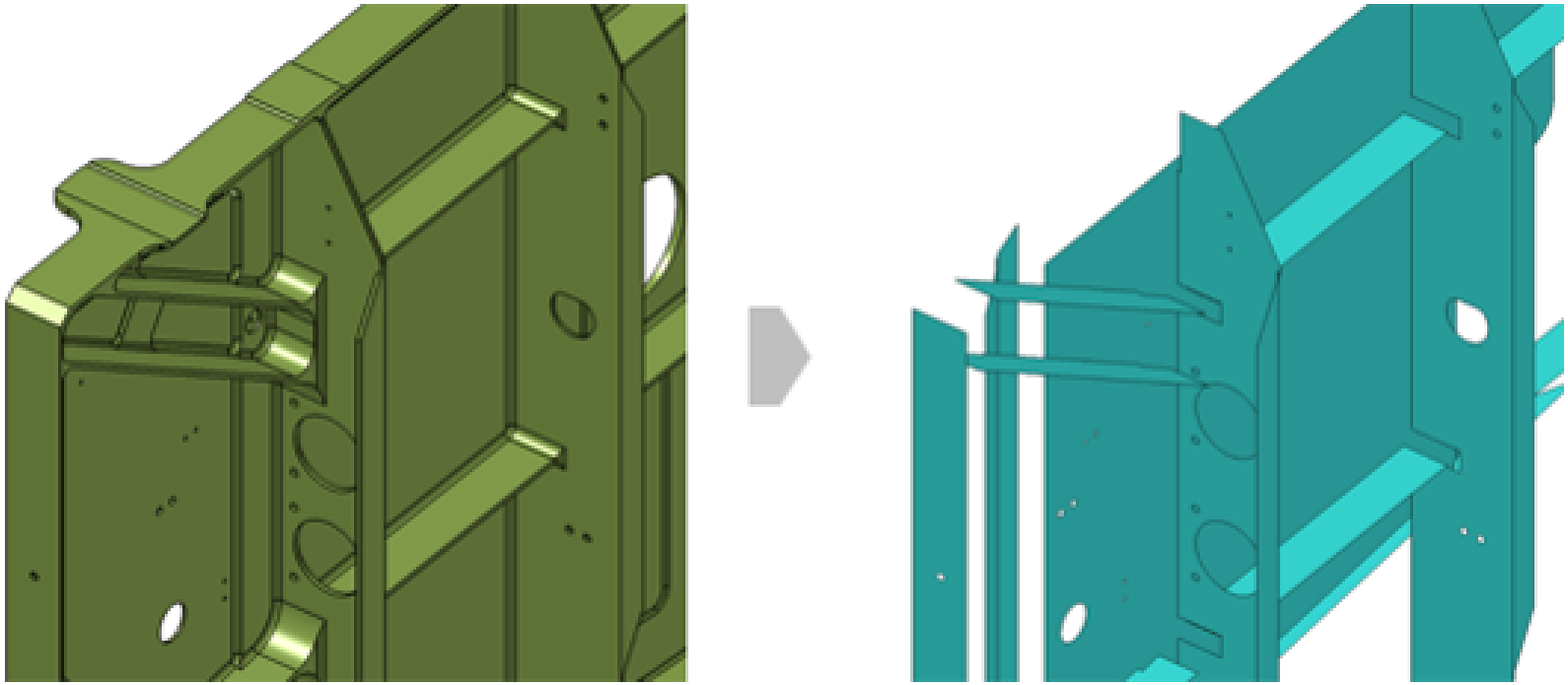
Getting Midsurface

- Going on for decades ...
- Manually by offsetting and stitching, initially
- Many CAD-CAE packages give automatic option, but ...



YHK

Look at the output

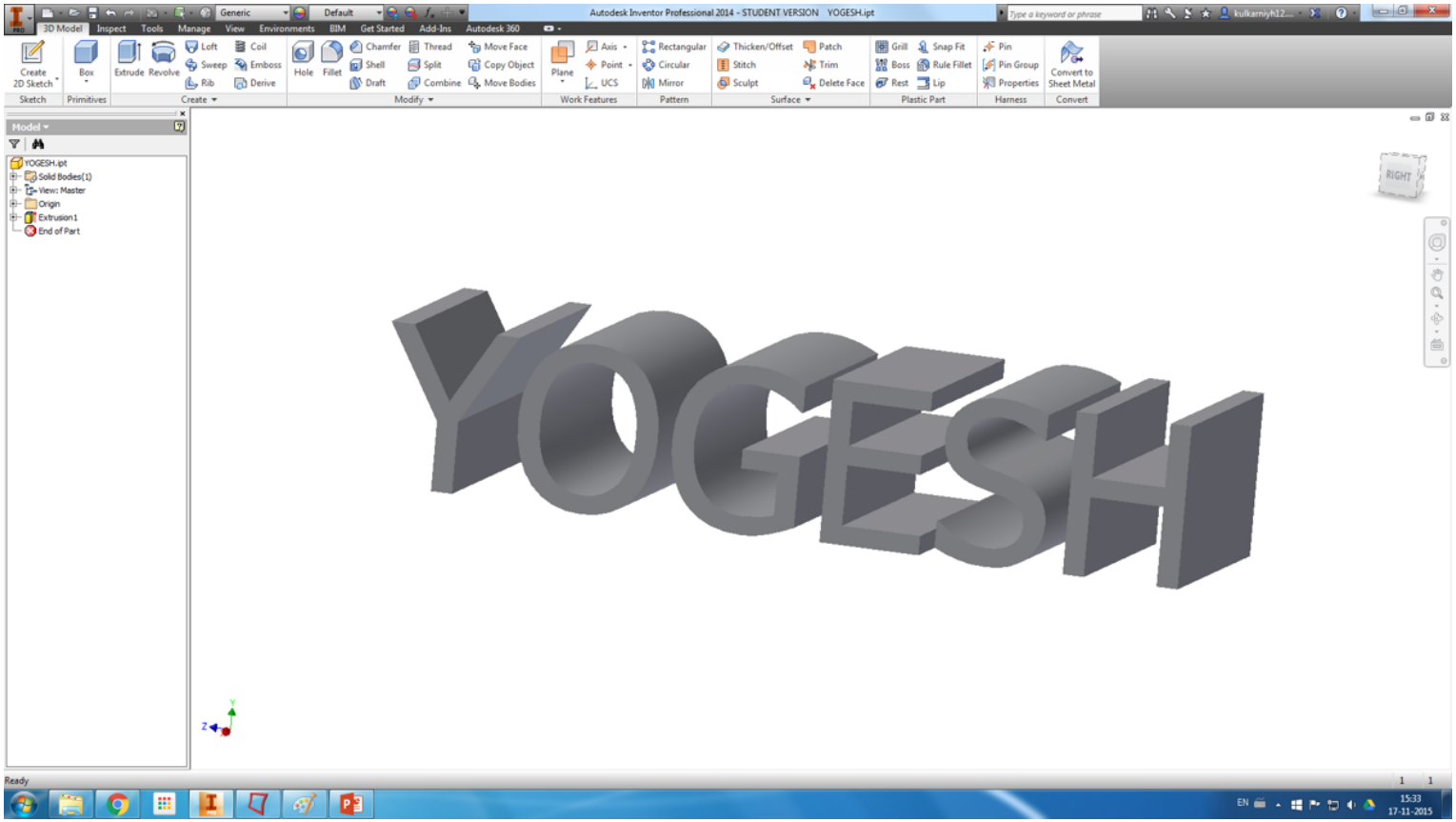


Can't tolerate gaps

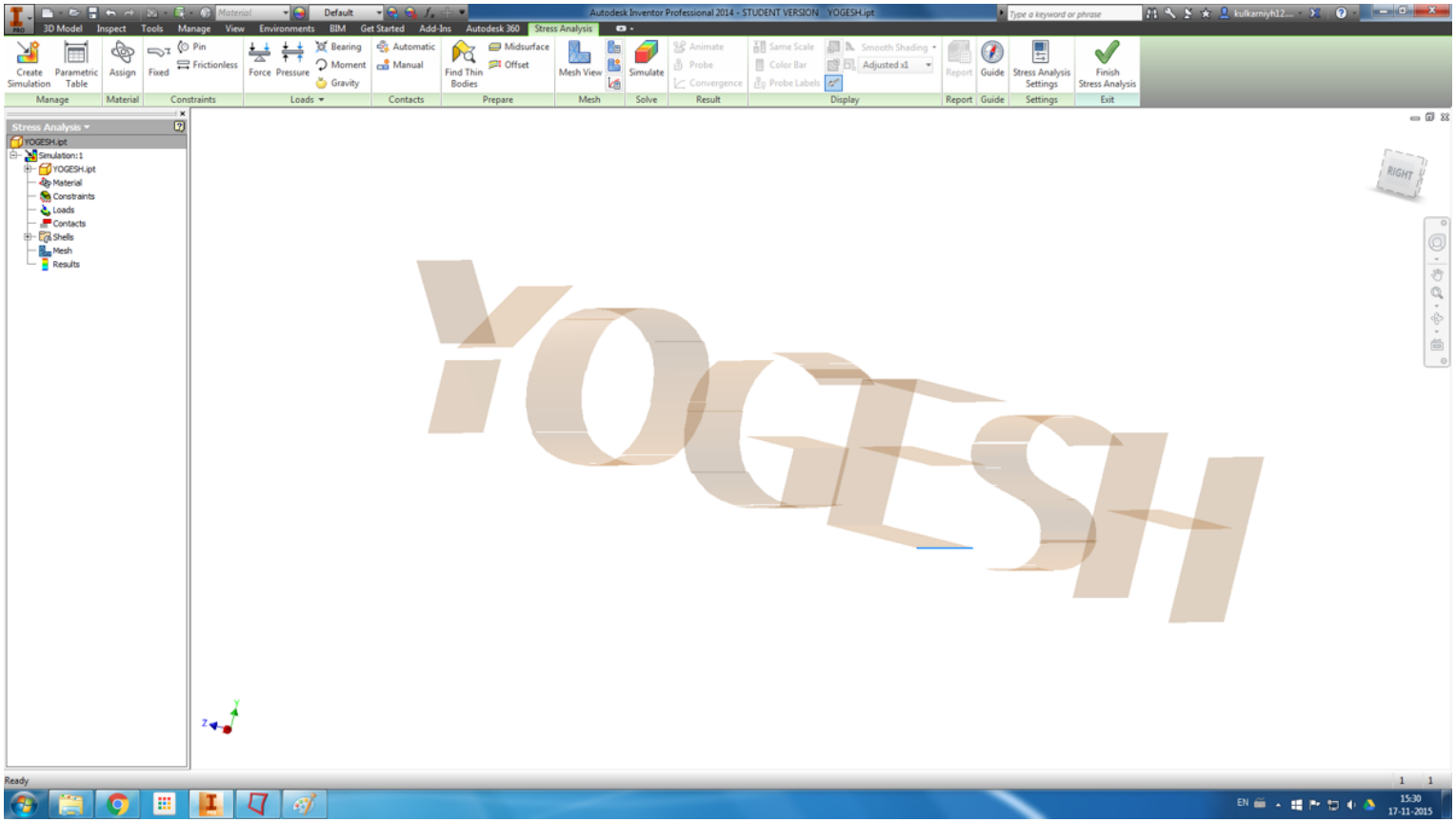
- We have thickness sampling,
- To recreate-represent the original shape
- Input and output difference not desirable

YHK

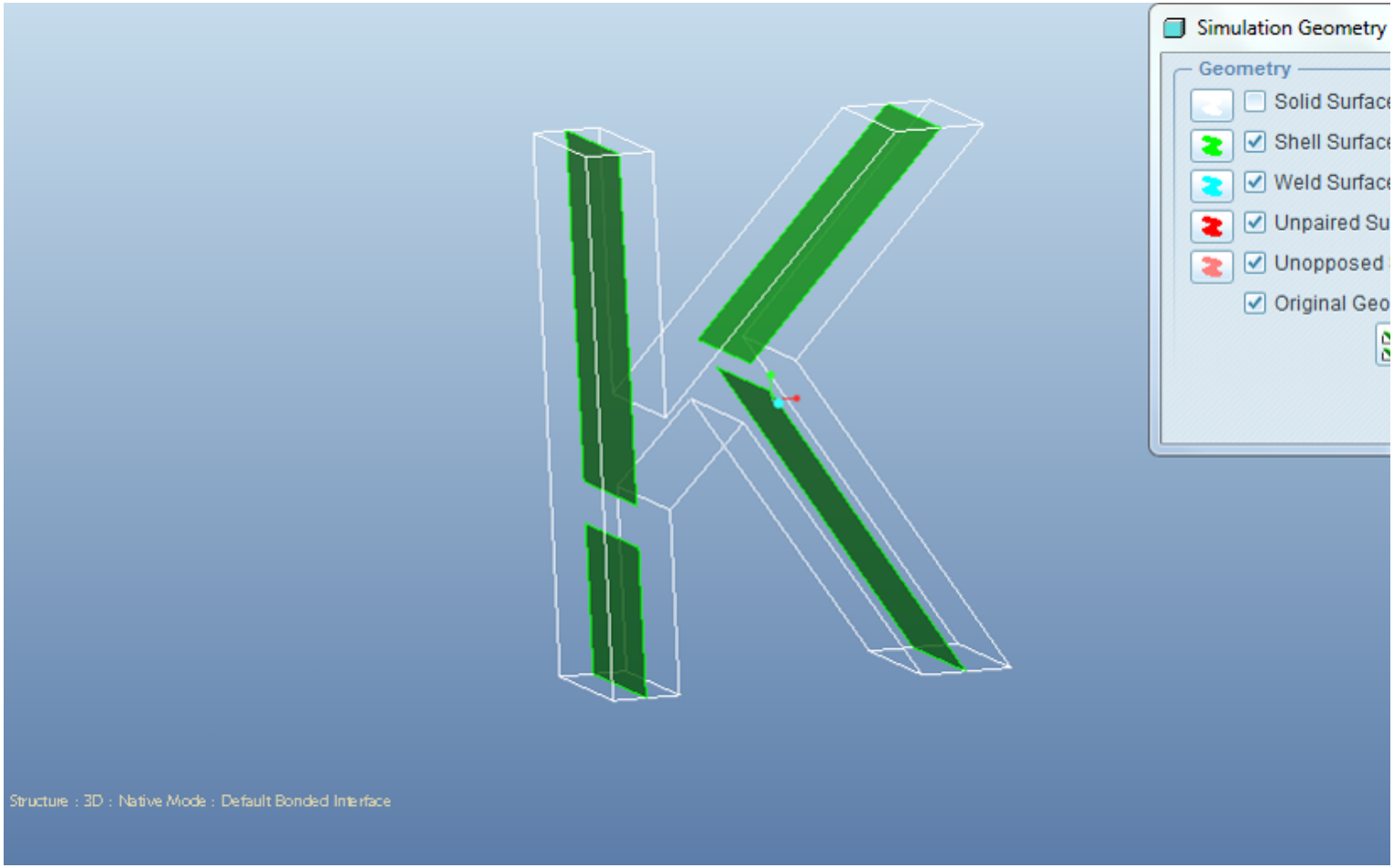
For a simple model like



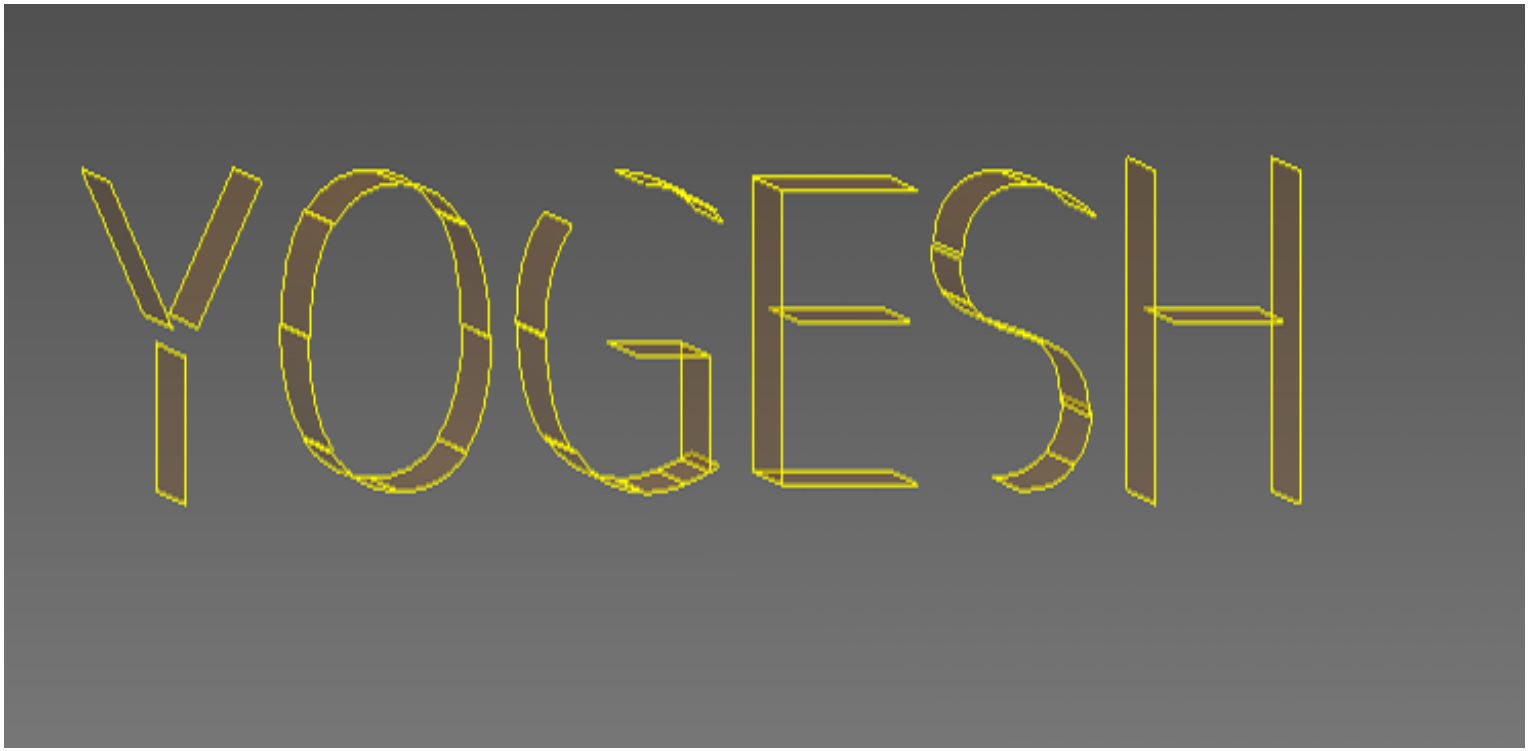
You get


Autodesk Inventor Professional 2014 - STUDENT VERSION YOGESH.ipt
Stress Analysis
YOGESH.ipt
Simulation:1
Material
Constraints
Loads
Contacts
Shells
Mesh
Results
RIGHT
YOGESH
Ready

For a far simpler shape



Current Quality

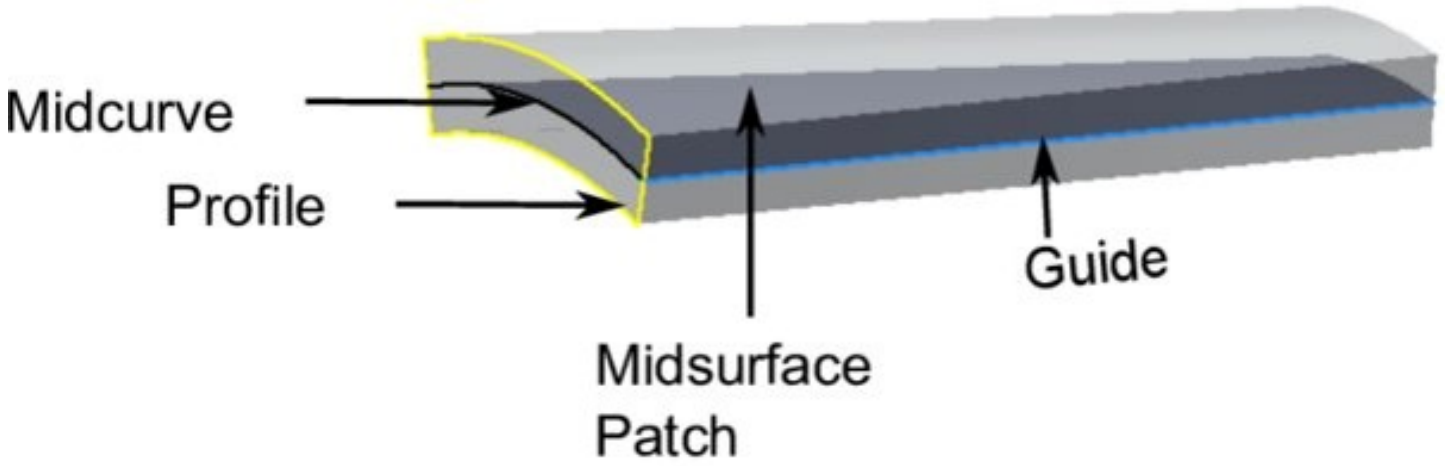


- Errors take weeks to correct for complex parts.
- But still preferred, due to vast savings time
- From Days to hours ...

YHK

Midsurface Computation

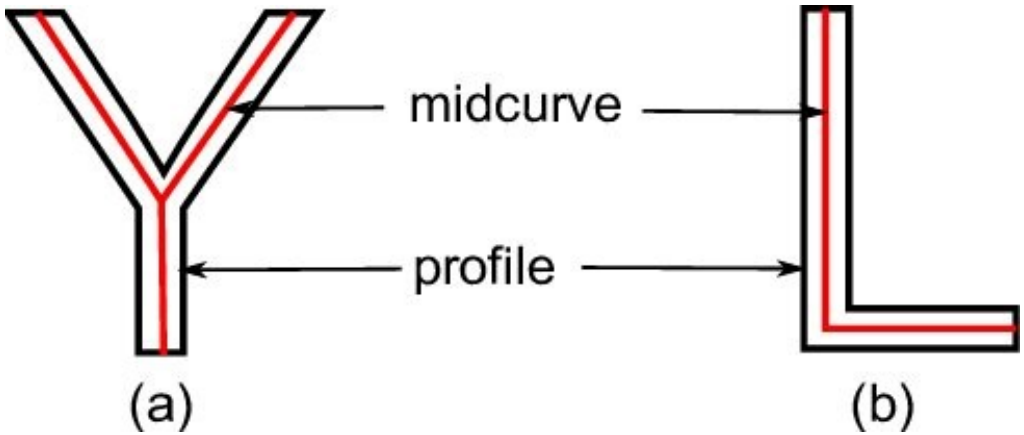
- Midsurface of a Patch is Midcurve of its profile extruded.
- So, it boils down to computing 1D midcurve of a 2D profile



YHK

What is a Midcurve?

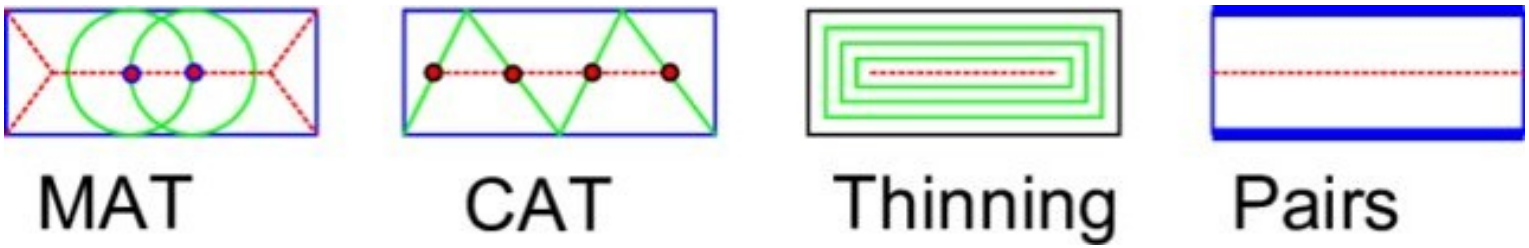
- Midsurface : From 3D thin Solid to 2D Surface
- Midcurve : From 2D Profile to 1D Curve



YHK

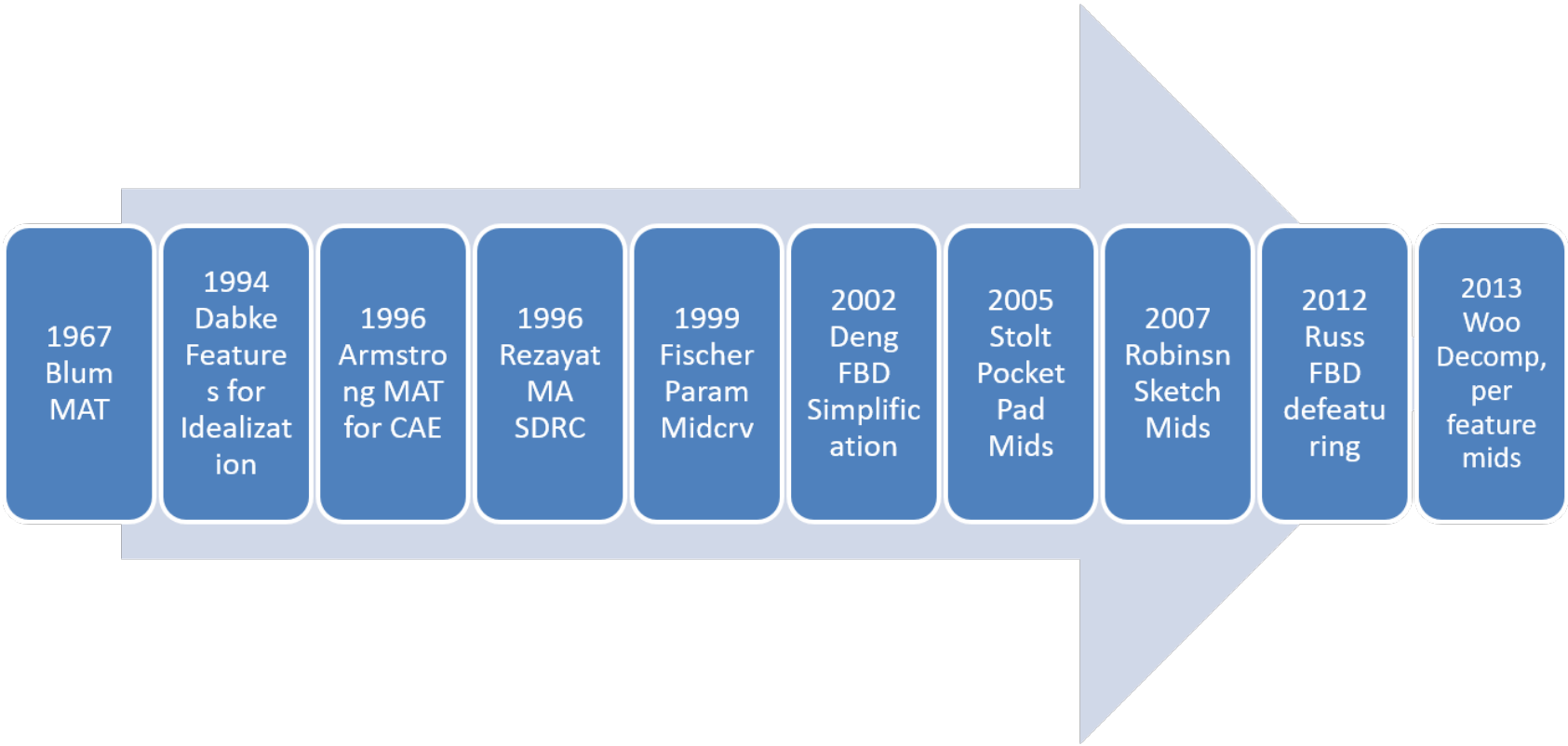
Many Approaches

- More than 6 decades of research...
- Most CAD-CAE packages...
- Rule-based!! Heuristic!! Case-by-case basis!!



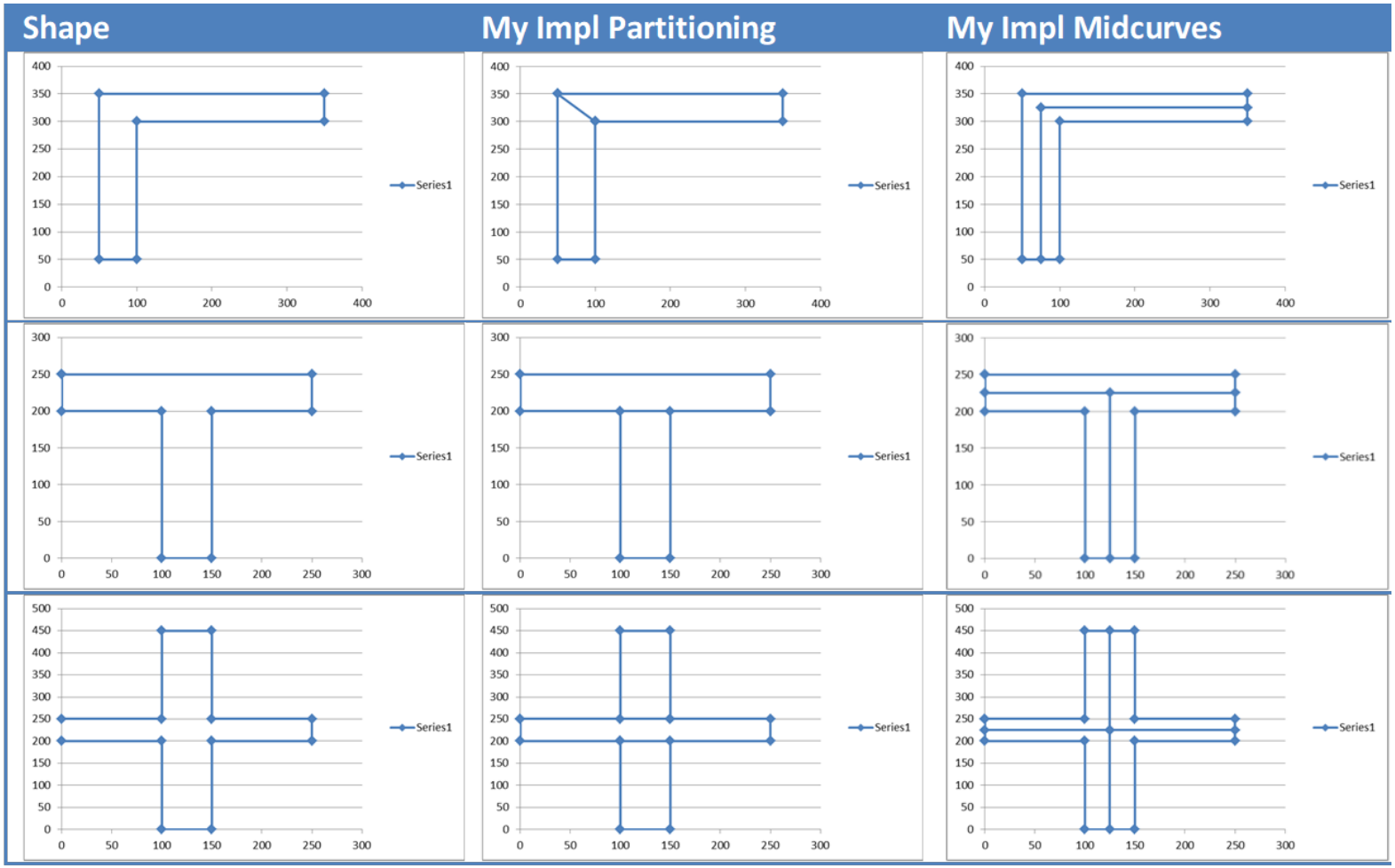
YHK

When-What?



YHK

2017: My PhD Work: Rule-based



Limitations

- Fully rule-based
- Need to adjust for new shapes
- So, not scalable



YHK

Idea



Can Neural Networks “learn” the dimension reduction transformation?

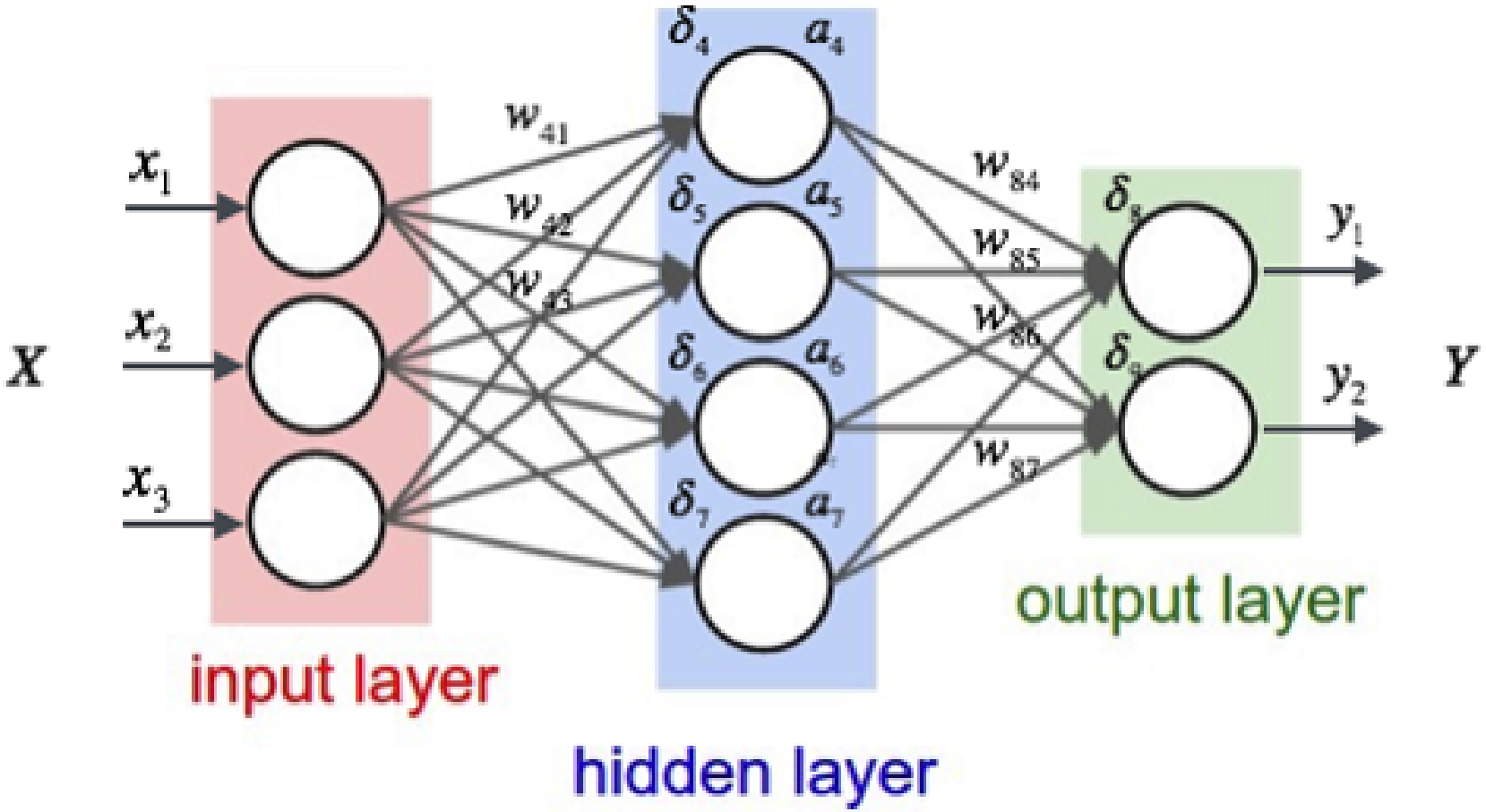
How?

- Supply lots of training data of profiles and their corresponding midcurves and train.
- Then given an unseen profile, can Neural Network compute a midcurve, mimicking the original profile shape?



YHK

Midcurve by Neural network



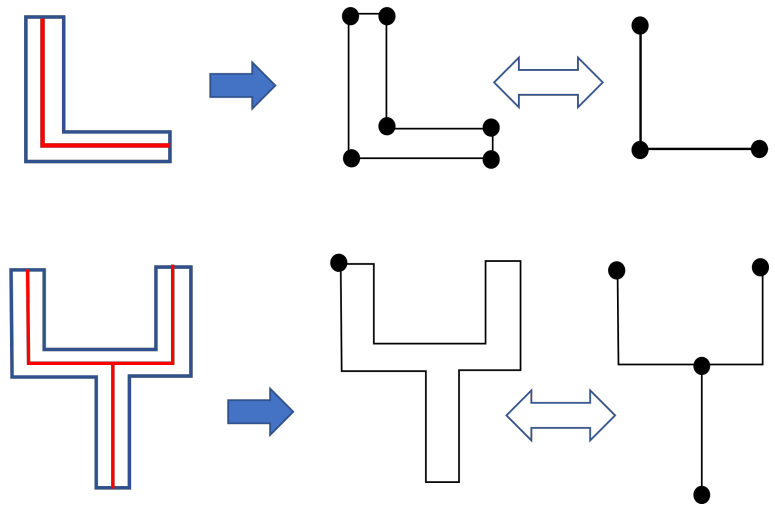
Midcurve : The Problem

- **Goal**: Given a 2D closed shape (closed polygon) find its midcurve (polyline, closed or open)
- **Input**: set of points or set of connected lines, non-intersecting, simple, convex, closed polygon
- **Output**: another set of points or set of connected lines, open/branched polygons possible

YHK

Midcurve : Graph 2 Graph (The Ideal)

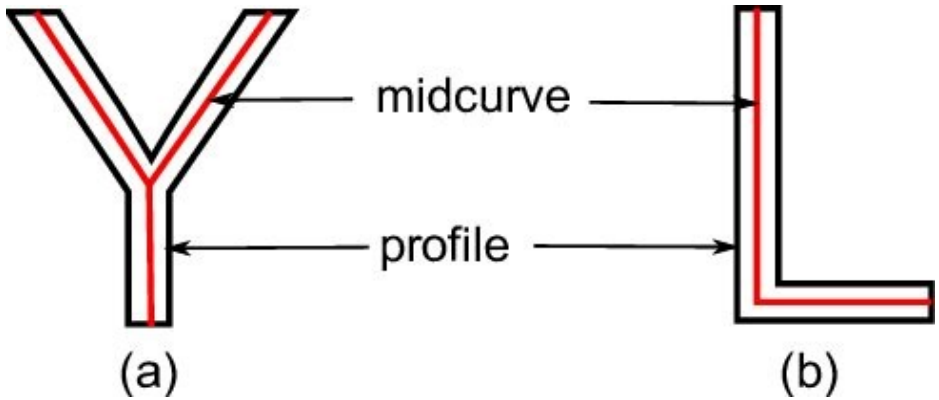
- **Ideal framing**: Input profile graph → Output midcurve graph
- Both graphs have **different topologies**: different node/edge counts, positions
- Standard node-prediction and link-prediction GNNs assume similar input/output topology; not suitable here
- Direct graph-to-graph translation requires a coarsening / dimension-reduction formulation
- Phase III of this work tackles this directly using a Graph Transformer



YHK

Midcurve == Dimension Reduction / Translation

- Like PCA, find the principal curve that *represents* the profile shape
- Left side (input): 2D Sketch Profile; Right side (output): 1D Midcurve
- **Not** an Auto-Encoder: input and output are *different* shapes
- Variable-length input/output makes padding unsafe in geometry (any value is valid)
- Analogous to **graph summarization**: reduce a large graph to a smaller graph preserving structural essence, like text summarization keeps the meaning

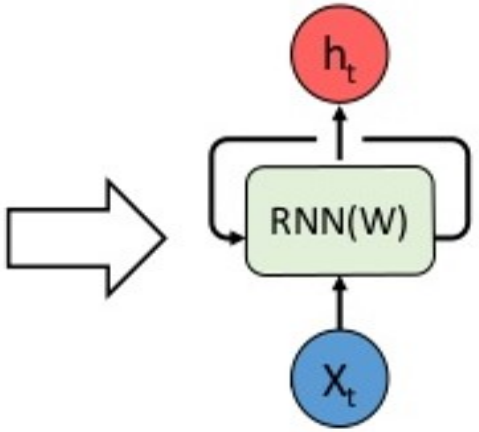


YHK

Variable Size Encoder Decoder

- Batches need fixed lengths
- Made fixed size by Padding.

Friendly	against	Scotland	at	Murray	.
Nadim	Ladki	<PAD>	<PAD>	<PAD>	<PAD>
AL-AIN	United	Arab	Emirates	<PAD>	<PAD>
ROME	1996-12	<PAD>	<PAD>	<PAD>	<PAD>
Two	goals	in	the	last	minutes



YHK

Why Padding Fails in Geometry

- OK for NLP, say Machine Translations, where padding values like "-1" can be added along with other words (vectors or indices)
- But in Geometry, its not OK.
- Because any value can represent a Valid Input, even though we don't want it to be the input.
- Not all shapes can be serialised as sequences: Y-shapes and concentric rings (O) cannot be traversed without lifting the pen, ruling out Seq2Seq models entirely

YHK

Three Phase Strategy

This trade-off motivates the **three-phase** strategy:

- **Phase I**: Raster Image → Image (encoder-decoder variants)
- **Phase II**: BRep JSON via LLMs, exact coords, current best performer
- **Phase III**: Geometry graph → graph, exact, native branches

YHK

A Twist to the problem

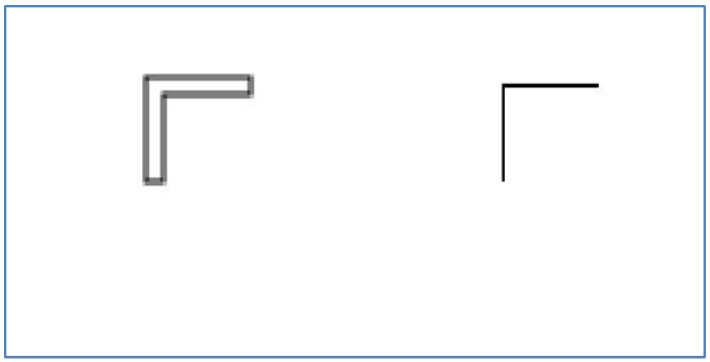


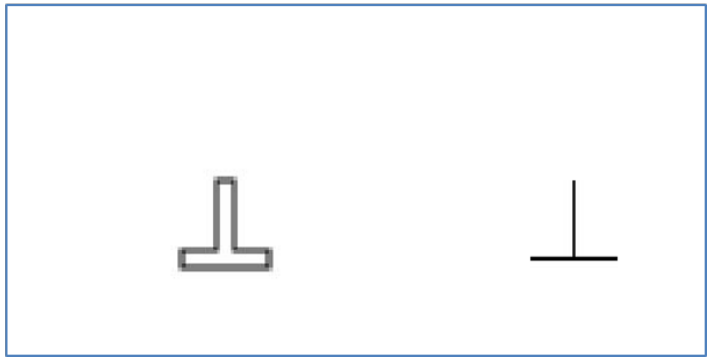
- Till we get good variable size encoder decoder network for geometry...
- Decided to convert this Sequence 2 Sequence problem as Image 2 Image problem.

YHK

A Twist to the problem

- Input: Black & White Image of 2D profile
- Output: Black & White Image of 1D midcurve





Solves ...

Problems of Geometric sequences

- Variable input/output sizes
- Loops need to be crossed
- Branches



YHK

Phase I: Raster Approximation

- Images address both the **representation** and **variable-size** issues
- Big dilution: True geometric shapes are *vector* data; images here are *raster*, so approximation has crept in
- Predicted output still needs post-processing to recover exact geometric (vector) form

YHK

For Deep Learning

- Need lots of data
- Had just few input output image pairs
- How to augment/populate large variations ...

YHK

Rasterization: Geometry to Image

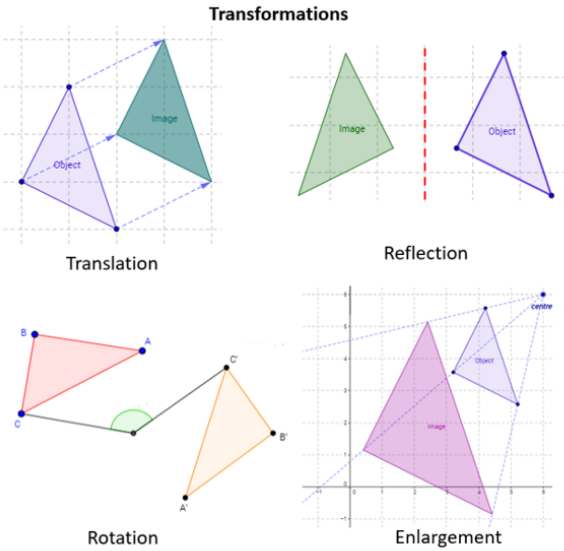
- Raw data is in vector format (list of (x, y) points)
- Converted to fixed-size 100×100 bitmap via DrawSVG rasterization





YHK

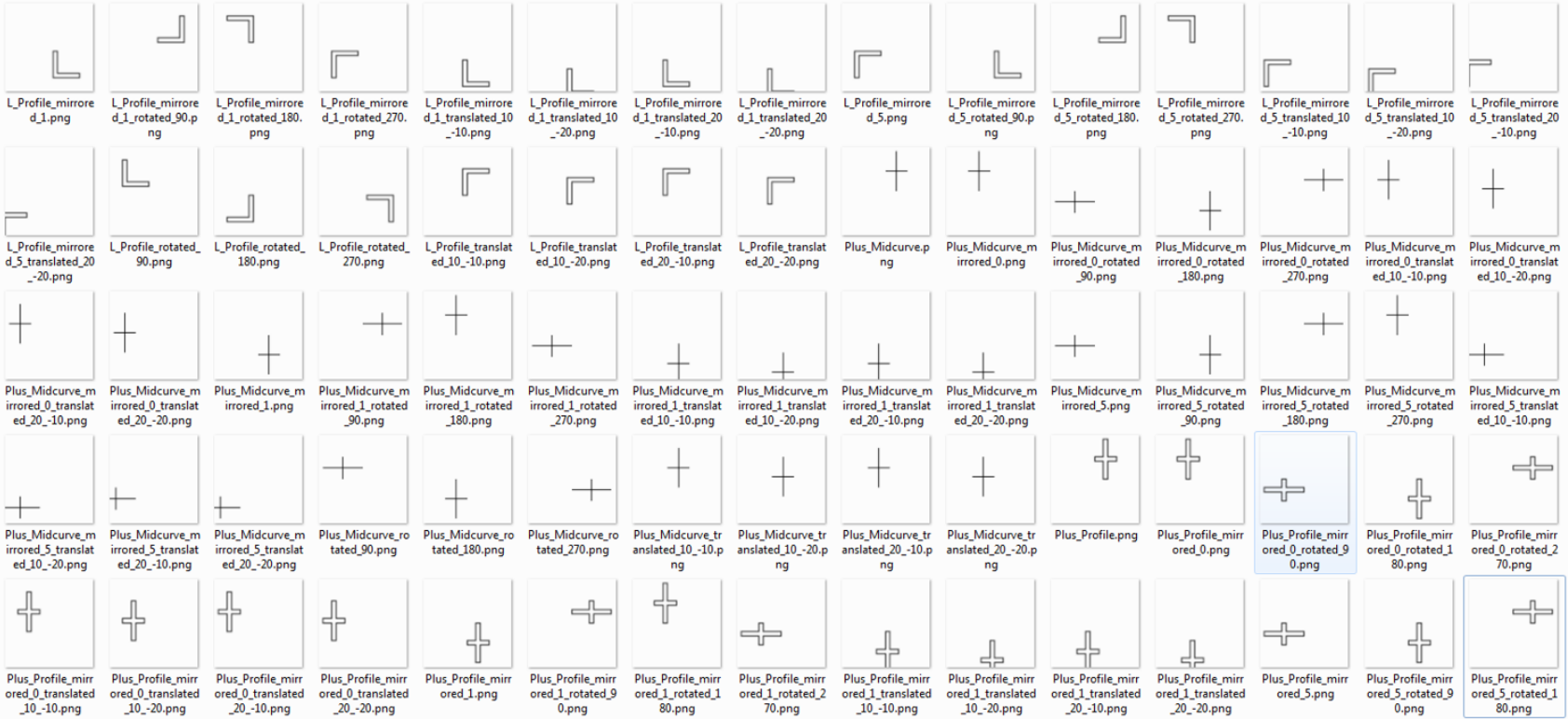
Augmentation: Geometric Variations



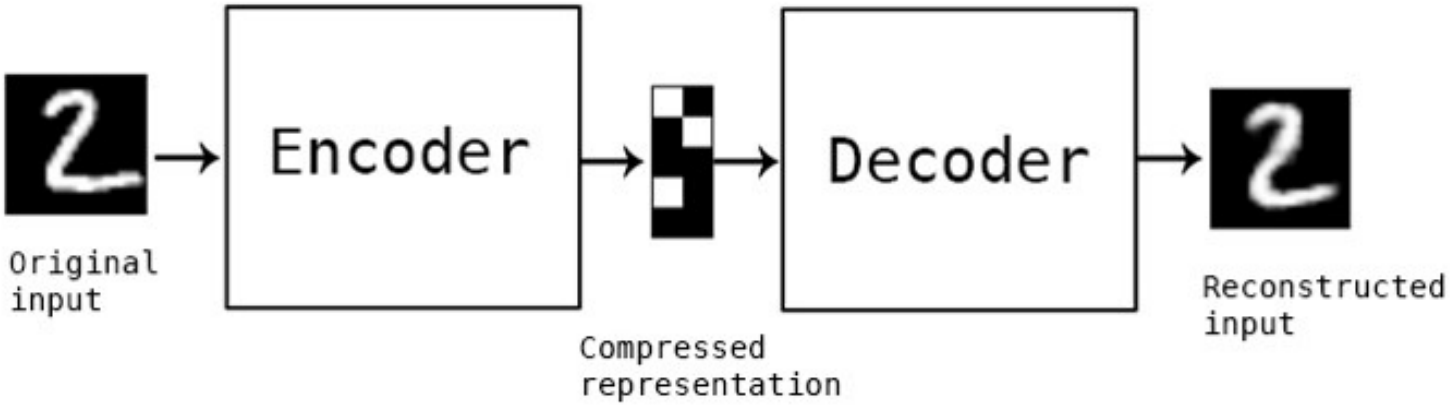
- Base shapes: I, L, Plus, T
- Operations applied to both profile and midcurve:
  - Translated
  - Rotated
  - Mirrored
  - Mirrored + Translated
  - Mirrored + Rotated
- Total: 896 image pairs

YHK

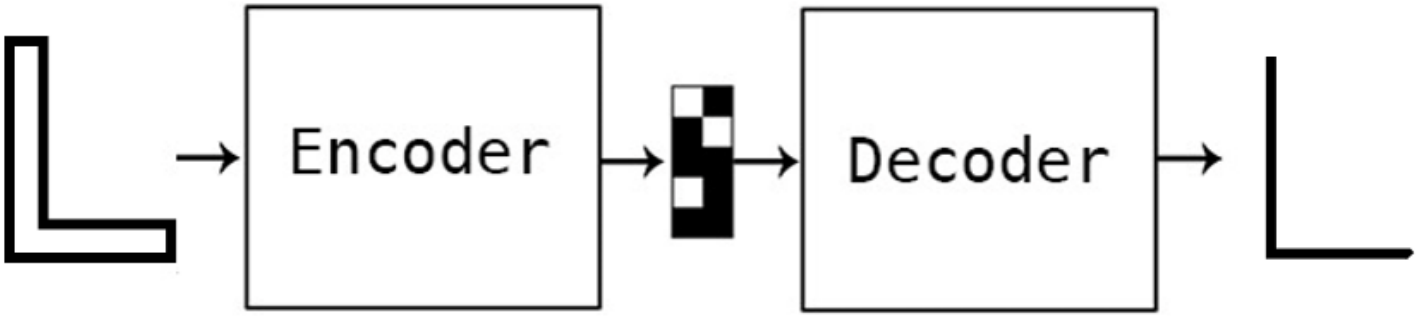
Training Data Samples



Reuse Image Encoder Decoder



For Dimension Reduction



Phase II: Text as a Medium for Geometry

- Phase I approximates geometry as pixel images: raster dilution
- Phase II takes a different route: serialize geometry as **structured JSON text**
- LLMs can read and write JSON naturally: frame midcurve as **machine translation**:
  - Input: Profile geometry $\rightarrow$ BRep JSON text
  - Output: Midcurve geometry $\leftarrow$ predicted BRep JSON text
- **Advantage over Phase I**: output is *exact point coordinates*, not noisy pixels: no raster post-processing needed
- **Trade-off**: LLMs are probabilistic; output may occasionally need syntactic repair
- Models explored: Qwen2.5-7B, Gemma, Mistral (QLoRA fine-tuning); Nemotron-Mini-4B; few-shot prompting

YHK

BRep: Boundary Representation in JSON

Geometry encoded as a 3-level hierarchy (2D adaptation of solid-modelling B-rep):

- **Points**: ordered list of (x, y) coordinates
- **Lines**: pairs of point indices: encodes *connectivity*
- **Segments**: chains of line indices: encodes *continuous open curves*; **multiple Segments** represent junctions in T- and Plus-shapes

Key invariant: only *coordinates* change under geometric transforms; topology (Lines, Segments) stays fixed

I-shape Profile:

`Points:` $(5, 5), (10, 5), (10, 20), (5, 20)$
`Lines:` $[0, 1], [1, 2], [2, 3], [3, 0]$

`Segs:` $[0, 1, 2, 3]$

I-shape Midcurve:

`Points:` $(7.5, 5), (7.5, 20)$
`Lines:` $[0, 1]$

`Segs:` $[0]$

T-shape Midcurve (branched):

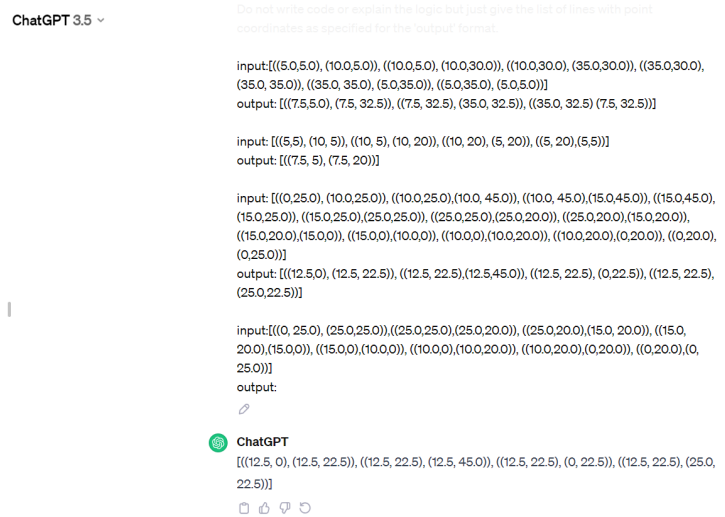
`Segs:` $[0], [1], [2]$ (3 arms = 3 Segments)

Sequences cannot encode this; BRep can.

YHK

Phase II: How LLMs Predict Midcurves

- **Data**: 993 BRep examples: 4 base shapes (I, L, T, Plus) × geometric augmentations (rotate, scale, translate, mirror)
- **Framing**: text-to-text machine translation
  - Input: `Profile_brep` JSON string
  - Output: `Midcurve_brep` JSON string
- **Sub-approaches**:
  - Few-shot prompting (ChatGPT, Bard, Claude)
  - QLoRA fine-tuning (4-bit quantised Low-Rank Adaptation, trains <1% of params): Qwen2.5-7B, Gemma, Mistral
  - Nemotron-Mini-4B: HF SFTTrainer / Unsloth / few-shot



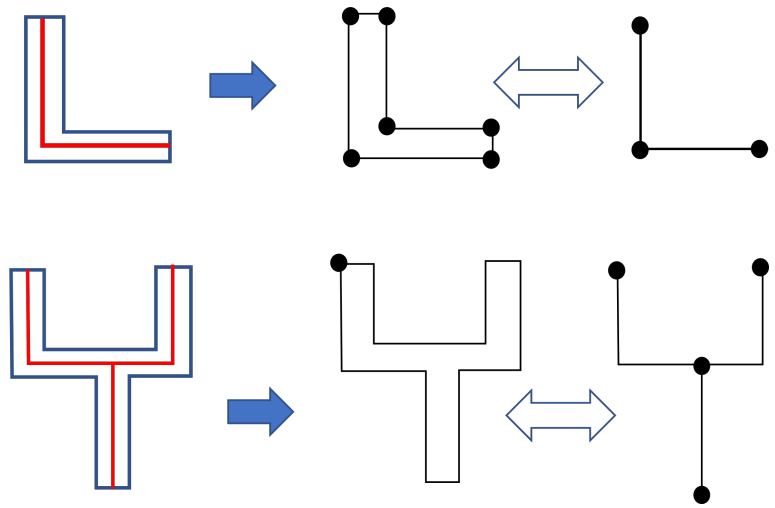
Phase II: Results

- **Qwen2.5-7B (QLoRA fine-tuning)**: current best performer:
  - MAE = 0.78, RMSE = 1.24, Hausdorff distance = 2.1
  - **PSR (Profile Shape Reconstruction) = 98%** on 100-sample test set
- **Nemotron-Mini-4B (2-shot, no fine-tuning)**:
  - PSR = 85.7%, topology accuracy = 0.83
  - Fits in ∼2 GB VRAM: deployable on consumer hardware
- **Key advantage**: output is exact *point coordinates*: no pixel clean-up
- **Limitation**: branched shapes (T, Plus) require well-formed multi-Segment JSON output

YHK

Phase III: Back to Pure Geometry: Graph to Graph

- Recall the ideal: input profile graph → output midcurve graph
- Phase III implements this directly: **no pixels, no text tokens**
- **Input**: polygon as graph: nodes = corner points with (x, y); edges = adjacent corners
- **Output**: midcurve as graph: different node/edge counts and positions
- **Advantage**: exact coordinates, native branch handling, zero post-processing
- Branches (T, Plus) are naturally captured in the graph topology



YHK

Phase III: Graph Transformer Architecture

- **Core challenge**: graph-to-graph translation with *different* input/output topology: not standard node/link prediction
- **Non-auto-regressive Graph Transformer** (trained from scratch):
  - LaplacianPE positional encoding → TransformerConv layers → TopKPooling
  - TopKPooling acts as graph *coarsening*: the geometric dimension-reduction step
  - Loss: Chamfer distance (point-cloud matching) + BCE (adjacency matrix)
- **Second variant**: pretrained Graphormer (HuggingFace) fine-tuned in two phases (frozen head → joint fine-tuning)
- Operates on raw (x, y) coordinates: no rasterization, no JSON encoding
- Status: implemented, evaluation ongoing

YHK

Three Phases: A Progression Toward the Ideal

	Phase I Image-based	**Phase II** LLM / Text	**Phase III** Geometry
Medium	Bitmap (raster)	BRep JSON text	Graph (x, y)
Exact coords	No (pixels)	Yes	Yes
Branches	No	Yes (Segments)	Yes (native)
Best model	CNN encoder-decoder	Qwen2.5-7B QLoRA	Graph Transformer
Key metric	Visual quality	PSR = 98%	Ongoing eval.
Status	Fully impl.	Best performer	Implemented

YHK

Conclusion

- Midcurve is a **geometric dimension-reduction** problem: 2D profile → 1D skeleton (graph → smaller graph)
- **Phase I** (image-based): encoder-decoder on raster images — feasible; CNN encoder-decoder is the best primary variant; rasterization trade-off limits precision
- **Phase II** (LLM / text-based): BRep JSON + LLM fine-tuning: current best performer: PSR = 98%, exact coordinates, branching via Segments
- **Phase III** (geometry-based): Graph Transformer on polygon graphs: the ideal formulation; no approximation, native branch handling; evaluation ongoing
- **Next frontier**: generalise to arbitrary 2D thin polygons; extend the graph-to-graph paradigm toward 3D midsurface computation

Graph-to-Graph is the ultimate goal for MidcurveNN.

YHK

LLMs for Midcurve Implementation

Phase II: Key Idea



Can Large Language Models “learn” the dimension reduction transformation?

Proposed Approach

Text to Text Transformation Learning:

- Geometric Representation: A text-based representation of the geometry/graph/network will be explored to leverage Natural Language Processing (NLP) techniques.
- Existing Methods: The paper (Fatemi, Halcrow, and Bryan 2023) surveys several text-based representations for graph data, but none appear specifically suited for geometric shapes.
- Proposed Approach: A geometry representation similar to 2D Boundary Representation (B-rep) will be utilized, adapting the concept from 3D to 2D.

YHK

Representation of Geometry

- One limitation of 2D geometry-shape, represented as a sequential list of points, is that we can not represent line intersections or concentric loops.
- To address this a more comprehensive structure has been proposed which is based on the corresponding 3D structure popular in Solid Modeling, called Brep (Boundary Representation).

YHK

2D Brep Representation

Leverage a geometry representation similar to that found in 3D B-rep (Boundary representation), but in 2D. It can be shown as:

```
{
 'ShapeName': 'I',
 'Profile': [(5.0, 5.0), (10.0, 5.0), (10.0, 20.0), (5.0, 20.0)],
 'Midcurve': [(7.5, 5.0), (7.5, 20.0)],
 'Profile_brep': {
    'Points': [(5.0, 5.0), (10.0, 5.0), (10.0, 20.0),(5.0, 20.0)], # list of
     (x,y) coordinates
    'Lines': [[0, 1], [1, 2], [2, 3], [3, 0]], # list of point ids (ie index
     in the Points list)
                'Segments': [[0, 1, 2, 3]] # list of line ids (ie index in
     Lines list)
    },
 'Midcurve_brep': {
    'Points': [(7.5, 5.0), (7.5, 20.0)],
    'Lines': [[0, 1]],
                'Segments': [[0]]
    },
}
```

YHK

Data

ShapeName	Profile	Midcurve	Profile_brep	Midcurve_brep
I	[[5.0, 5.0], [10.0, 5.0], [10.0, 20.0], [5.0, 20.0]]	[[7.5, 5.0], [7.5, 20.0]]	{"Points": [[5.0, 5.0], [10.0, 5.0], [10.0, 20.0], [5.0, 20.0]], "Lines": [[0, 1], [1, 2], [2, 3], [3, 0]], "Segments": [[0, 1, 2, 3]]}	{"Points": [[7.5, 5.0], [7.5, 20.0]], "Lines": [[0, 1]], "Segments": [[0]]}
L	[[5.0, 5.0], [10.0, 5.0], [10.0, 30.0], [35.0, 30.0], [35.0, 35.0], [5.0, 35.0]]	[[7.5, 5.0], [7.5, 32.5], [35.0, 32.5]]	{"Points": [[5.0, 5.0], [10.0, 5.0], [10.0, 30.0], [35.0, 30.0], [35.0, 35.0], [5.0, 35.0]], "Lines": [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 0]], "Segments": [[0, 1, 2, 3, 4, 5]]}	{"Points": [[7.5, 5.0], [7.5, 32.5], [35.0, 32.5]], "Lines": [[0, 1], [1, 2]], "Segments": [[0, 1]]}
Plus	[[0.0, 25.0], [10.0, 25.0], [10.0, 45.0], [15.0, 45.0], [15.0, 25.0], [25.0, 25.0], [25.0, 20.0], [15.0, 20.0], [15.0, 0.0], [10.0, 0.0], [10.0, 20.0], [0.0, 20.0]]	[[12.5, 0.0], [12.5, 22.5], [12.5, 45.0], [0.0, 22.5], [25.0, 22.5]]	{"Points": [[0.0, 25.0], [10.0, 25.0], [10.0, 45.0], [15.0, 45.0], [15.0, 25.0], [25.0, 25.0], [25.0, 20.0], [15.0, 20.0], [15.0, 0.0], [10.0, 0.0], [10.0, 20.0], [0.0, 20.0]], "Lines": [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 6], [6, 7], [7, 8], [8, 9], [9, 10], [10, 11], [11, 0]], "Segments": [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]]}	{"Points": [[12.5, 0.0], [12.5, 22.5], [12.5, 45.0], [0.0, 22.5], [25.0, 22.5]], "Lines": [[0, 1], [4, 1], [2, 1], [3, 1]], "Segments": [[0], [1], [2], [3]]}
T	[[0.0, 25.0], [25.0, 25.0], [25.0, 20.0], [15.0, 20.0], [15.0, 0.0], [10.0, 0.0], [10.0, 20.0], [0.0, 20.0]]	[[12.5, 0.0], [12.5, 22.5], [25.0, 22.5], [0.0, 22.5]]	{"Points": [[0.0, 25.0], [25.0, 25.0], [25.0, 20.0], [15.0, 20.0], [15.0, 0.0], [10.0, 0.0], [10.0, 20.0], [0.0, 20.0]], "Lines": [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 6], [6, 7], [7, 0]], "Segments": [[0, 1, 2, 3, 4, 5, 6, 7]]}	{"Points": [[12.5, 0.0], [12.5, 22.5], [25.0, 22.5], [0.0, 22.5]], "Lines": [[0, 1], [1, 2], [3, 1]], "Segments": [[0], [1], [2]]}
I_scaled_2	[[10.0, 10.0], [20.0, 10.0], [20.0, 40.0], [10.0, 40.0]]	[[15.0, 10.0], [15.0, 40.0]]	{"Points": [[10.0, 10.0], [20.0, 10.0], [20.0, 40.0], [10.0, 40.0]], "Lines": [[0, 1], [1, 2], [2, 3], [3, 0]], "Segments": [[0, 1, 2, 3]]}	{"Points": [[15.0, 10.0], [15.0, 40.0]], "Lines": [[0, 1]], "Segments": [[0]]}
L_scaled_2	[[10.0, 10.0], [20.0, 10.0], [20.0, 60.0], [70.0, 60.0], [70.0, 70.0], [10.0, 70.0]]	[[15.0, 10.0], [15.0, 65.0], [70.0, 65.0]]	{"Points": [[10.0, 10.0], [20.0, 10.0], [20.0, 60.0], [70.0, 60.0], [70.0, 70.0], [10.0, 70.0]], "Lines": [[0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 0]], "Segments": [[0, 1, 2, 3, 4, 5]]}	{"Points": [[15.0, 10.0], [15.0, 65.0], [70.0, 65.0]], "Lines": [[0, 1], [1, 2]], "Segments": [[0, 1]]}

YHK

Data Explanation

Column information is as below:

- ShapeName (text): name of the shape. Just for visualization/reports.
- Profile (text): List of Points coordinates (x,y) representing outer profile shape, typically closed.
- Midcurve (text): List of Points coordinates (x,y) representing inner midcurve shape, typically open.
- Profile_brep (text): Dictionary in Brep format representing outer profile shape, typically closed.
- Midcurve_brep (text): Dictionary in Brep format representing inner midcurve shape, typically open.

Each Segment is a continuous list of lines. In case of, say 'Midcurve-T', as there is an intersection, we can treat each line in a separate segment. In the case of 'Profile O', there will be two segments, one for outer lines and another for inner lines. Each line is a list of points, for now, linear. Points is a list of coordinates (x,y), later can be (x,y,z).

YHK

Data Resolution

- Once we have this Brep representations of both, profile and the corresponding midcurve, in the text form, then we can try various machine translation approaches or LLM based fine tuning or few-shots prompt engineering.
- One major advantage of text based method over image based method is that image output still has stray pixels, cleaning which will be a complex task. But the text method has exact points. It may just give odd lines, which can be removed easily.

YHK

Prompt based techniques

Few Shots Prompt

```
You are a geometric transformation program that transforms input 2D polygonal
     profile to output 1D polyline profile. Input 2D polygonal profile is
     defined by set of connected lines with the format as: ...

Below are some example transformations, specified as pairs of 'input' and the
     corresponding 'output'. After learning from these examples, predict the
     'output' of the last 'input' specified.
Do not write code or explain the logic but just give the list of lines with
     point coordinates as specified for the 'output' format.

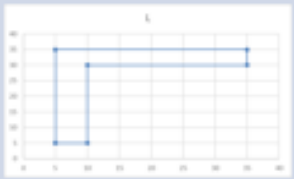
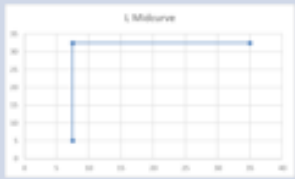
input:[((5.0,5.0), (10.0,5.0)), ... ((5.0,35.0), (5.0,5.0))]
output: [((7.5,5.0), (7.5, 32.5)), ... ((35.0, 32.5) (7.5, 32.5))]

input: [((5,5), (10, 5)), ...  (5, 20)), ((5, 20),(5,5))]
output: [((7.5, 5), (7.5, 20))]

:
input:[((0, 25.0), (25.0,25.0)),...  ((0,20.0),(0, 25.0))]
output:
```

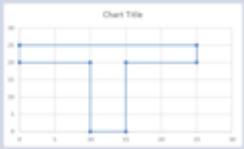
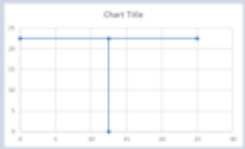
Few Shot: L-Shape Example

The first input example above represents 'L' shape (shown below) and the second is an 'I', whereas the 3rd is a 'Plus' sign shape.

Profile Data		Profile Picture	Midcurve Data		Midcurve Picture
5.0	5.0		7.5	5.0	
10.0	5.0		7.5	32.5	
10.0	30.0		35.0	32.5	
35.0	30.0		7.5	32.5	
35.0	35.0				
5.0	35.0				

Few Shot: T-Shape (Query)

The last shape for which LLM has been asked for the answer is actually a 'T' shape. The picture below shows the correct/actual answer as well.

Profile Data		Profile Picture	Midcurve Data		Midcurve Picture
0	25.0		12.5	0	
25.0	25.0		12.5	22.5	
25.0	20.0		25.0	22.5	
15.0	20.0		0	22.5	
15.0	0				
10.0	0				
10.0	20.0				
0	20.0				

LLM Raw Outputs

And the outputs computed by various LLMs (ChatGPT, Perplexity AI, Bard), along with the real/actual answer:

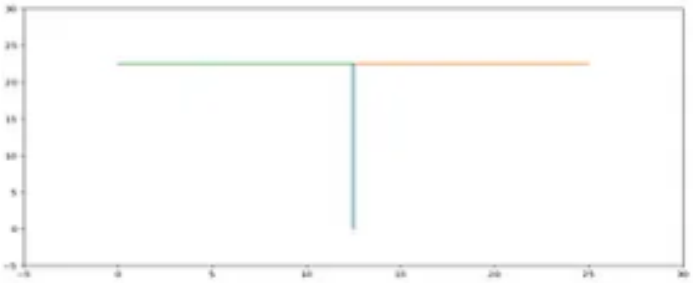
```
Actual: [((12.5,0), (12.5,22.5)), ((12.5,22.5),(25.0,22.5)),
     ((12.5,22.5),(0,22.5))]
ChatGPT: [((2.5, 0), (2.5, 22.5)), ((2.5, 22.5), (2.5, 45.0)), ((2.5, 22.5),
     (25.0, 22.5)), ((2.5, 22.5), (12.5, 22.5)), ((2.5, 22.5), (0, 22.5)),
     ((2.5, 22.5), (25.0, 22.5))]
Perplexity: [((12.5,0), (12.5, 22.5)), ((12.5, 22.5),(12.5,45.0)), ((12.5,
     22.5), (0,22.5)), ((12.5, 22.5), (25.0,22.5))]
Bard: [((12.5, 0), (12.5, 25.0)), ((12.5, 25.0), (25.0, 25.0)), ((25.0, 25.0),
     (25.0, 0))]
```

YHK

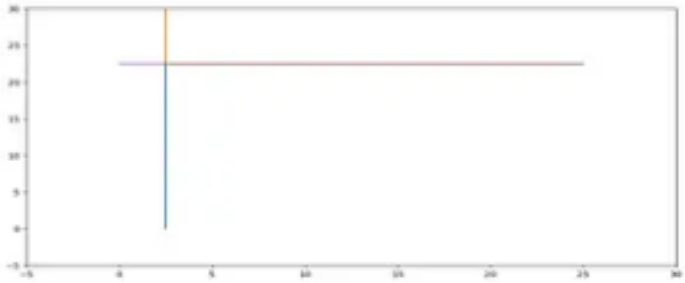
LLM Outputs: Visual Comparison

Visually here is how results from different LLMs look:

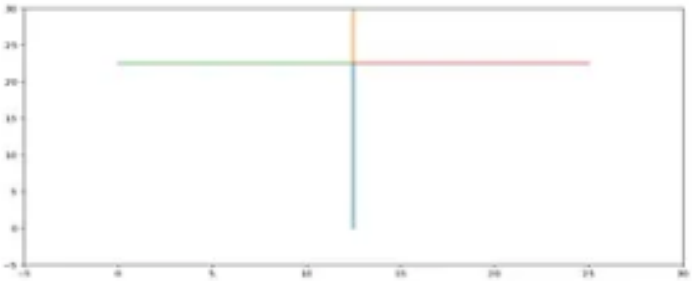
Actual/Expected



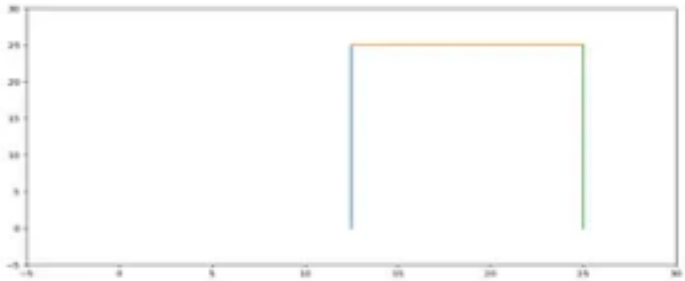
ChatGPT



Perplexity AI



Bard



Initial Results: Most Models Fail

Most models fail; only GPT-4 gives a correct answer. Results from various models:

- `llama 7B g4_0 ggml: (8, 17) & (64, 32)`: Wrong.
- `Gemini: [((8.33,5),(8.33, 22.5)), ((8.33, 22.5), (25,22.5)), ((8.33, 22.5), (0,25))]`: Wrong.
- `Hugging Chat: [((12.5, 0), (12.5, 22.5)), ((12.5, 22.5), (25.0, 22.5)), ((25.0, 22.5), (25.0, 25.0))]`: a bit wrong on the last line
- `GPT-4: [((12.5,0), (12.5,22.5)), ((12.5,22.5),(0,22.5)), ((12.5,22.5),(25.0,22.5))]` just change in sequence of lines, and that's inconsequential, so the answer is correct.

YHK

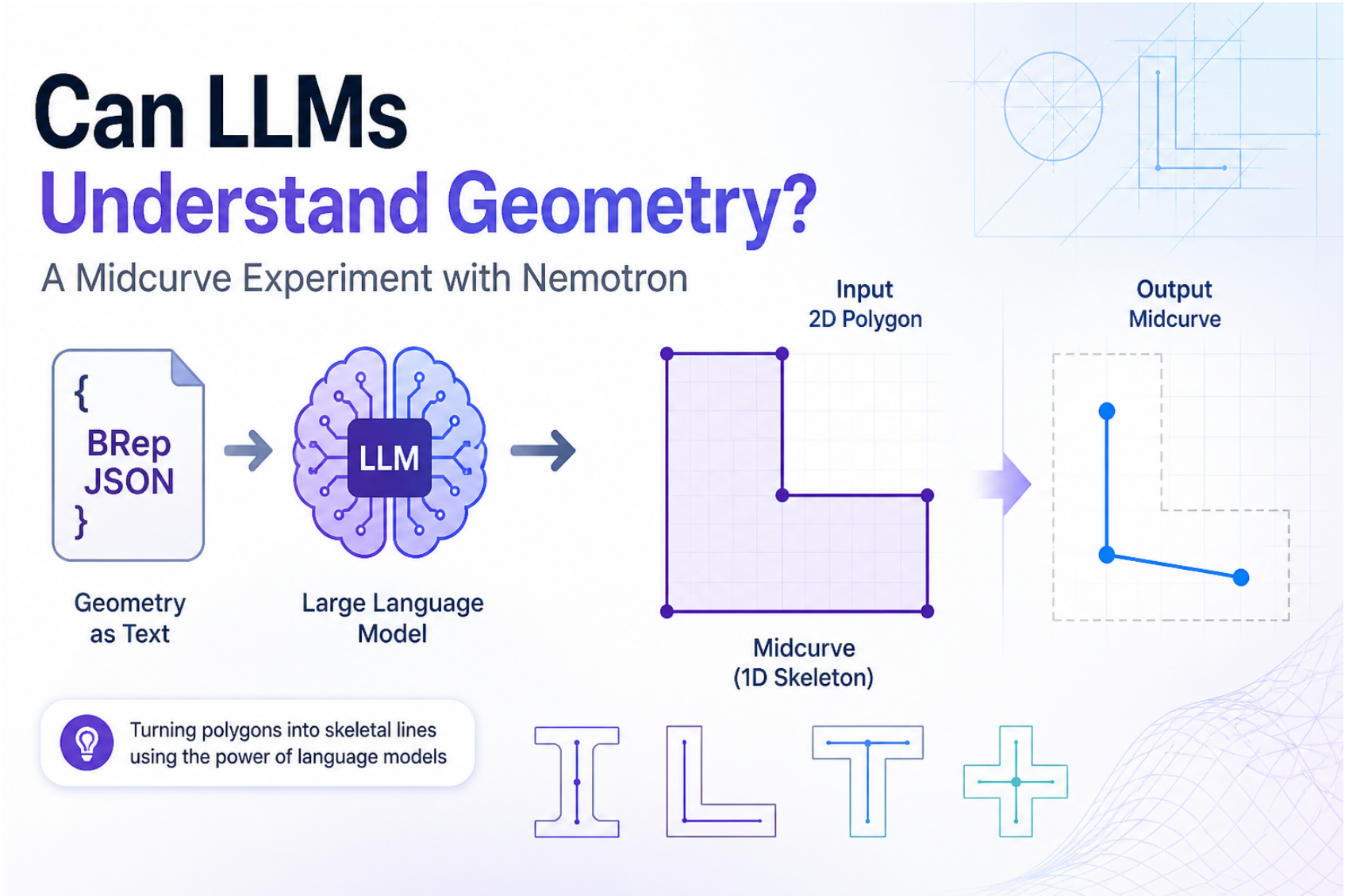
Interpretation: Summary

- **GPT-4**: correct answer — sequence-only representation with good prompting is sufficient for simple shapes
- **Smaller models (Llama 7B, Gemini, Bard)**: fail on coordinate arithmetic with raw sequence format
- **Key insight**: the flat line-sequence representation has limitations for complex shapes; a richer structure (BRep JSON) is needed
- **Solution**: switch to BRep JSON + fine-tuning $\Rightarrow$ Qwen2.5-7B achieves MAE $= 0.78$, PSR $= 98\%$

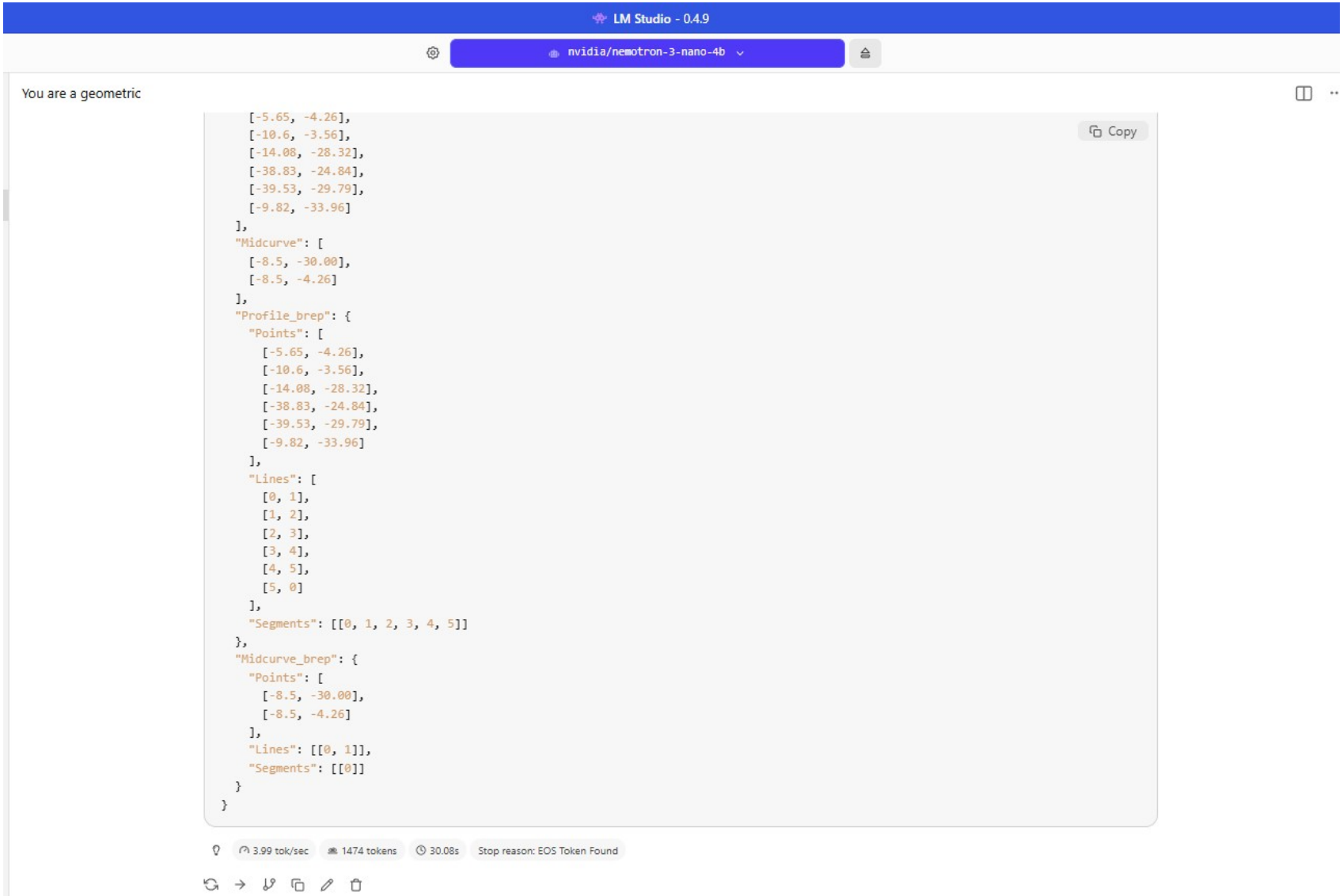
Legacy, leaked, not re-measured. Do not quote these figures.

YHK

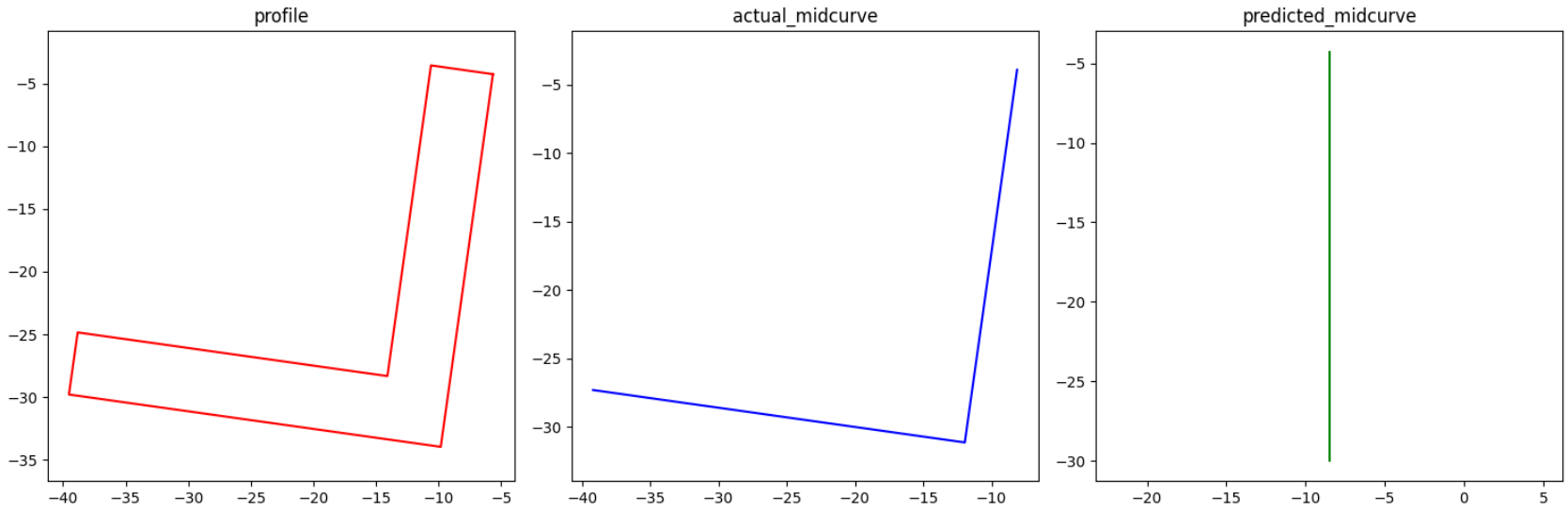
Nemotron-Mini-4B: NVIDIA's Compact Model



Nemotron: Few-Shot Prompt



Nemotron: Few-Shot Prompt Result



Predicted midcurve (red) vs ground truth (green)

Fine-tuning based techniques

Training Data

```
from datasets import load_dataset, Dataset, DatasetDict

base_url = "/content/drive/MyDrive/ImpDocs/Work/AICoach/Notebooks/data/"
dataset = load_dataset("csv", data_files={"train": base_url + "midcurve_llm_train.csv",
                                          "test": base_url + "midcurve_llm_test.csv",
                                          "validation": base_url + "midcurve_llm_val.csv"})
DatasetDict({
    train: Dataset({
        features: ['ShapeName', 'Profile', 'Midcurve', 'Profile_brep', 'Midcurve_brep'],
        num_rows: 793
    })
    test: Dataset({
        features: ['ShapeName', 'Profile', 'Midcurve', 'Profile_brep', 'Midcurve_brep'],
        num_rows: 99
    })
    validation: Dataset({
        features: ['ShapeName', 'Profile', 'Midcurve', 'Profile_brep', 'Midcurve_brep'],
        num_rows: 100
    })
})
```

Data Preprocessing

The dataset is used to fine-tune the base model called "Salesforce/codet5-small". Trained model is saved and used for inference on test samples.

```
from transformers import T5ForConditionalGeneration, AdamW, get_linear_schedule_with_warmup
import pytorch_lightning as pl

class CodeT5(pl.LightningModule):
    def __init__(self, lr=5e-5, num_train_epochs=15, warmup_steps=1000):
        super().__init__()
        self.model_name = "Salesforce/codet5-small"
        self.model = T5ForConditionalGeneration.from_pretrained(self.model_name)
        self.save_hyperparameters()

    def forward(self, input_ids, attention_mask, labels=None):
        outputs = self.model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        return outputs
        :
        :

model = CodeT5()

trainer = Trainer(max_epochs=5,accelerator="auto",# gpus=1,
                  default_root_dir="/content/drive/MyDrive/CodeT5/Notebooks/Checkpoints",
                  logger=wandb_logger,
                  callbacks=[early_stop_callback, lr_monitor])
trainer.fit(model)
```

YHK

Inferencing

The output predicted far away from the corresponding ground truth.

```
input_ids = tokenizer(test_example['Profile_brep'], return_tensors='pt').input_ids
outputs = model.generate(input_ids)

print("Ground truth:", test_example['Midcurve_brep'])
print("Generated Midcurve:", tokenizer.decode(outputs[0], skip_special_tokens=True))

Ground truth: "{\"Points\": [[−8.4, 3.28], [−20.68, −5.33]], \"Lines\": [[0, 1]], \"Segments\": [[0]]}"
Generated Midcurve: "{\"Points\": [[−8.83, 4.32], [−21.23
```

YHK

QLoRA Fine-Tuning (Efficient Approach)

- 4-bit quantised base model (Qwen / Gemma / Mistral 7B)
- LoRA adapters on query/value projections: only ∼1% params trained
- Instruction-formatted BRep JSON dataset (993 rows)

```
# QLoRA Fine-tuning (pseudocode)
base = AutoModelForCausalLM.from_pretrained(
        "Qwen/Qwen-7B-Chat", load_in_4bit=True)

lora = LoraConfig(r=64, lora_alpha=128,
        target_modules=["q_proj","v_proj"])
model = get_peft_model(base, lora)

def fmt(row):  # instruction format
    return f"Profile: {row['Profile_brep']}\nMidcurve:"

trainer = SFTTrainer(model, train_dataset=dataset,
            formatting_func=fmt, max_seq_length=512)
trainer.train()        # saves adapter weights only
model.save_pretrained("Midcurve-Qwen-Coder-v1")
```

YHK

Phase II: Results Comparison (Legacy)

Approach	Model	PSR (%)	MAE	Topo
Few-shot (seq)	GPT-4	∼90	N/A	∼0.75
Few-shot (seq)	Gemini / Llama 7B	<30	N/A	N/A
Fine-tune	CodeT5-small	∼50	N/A	N/A
QLoRA	**Qwen2.5-7B**	**98**	**0.78**	N/A
Few-shot (BRep)	Nemotron-Mini-4B	85.7	N/A	0.83 *(prelim.)*

- BRep JSON + QLoRA (Qwen2.5-7B): best results — MAE = 0.78, PSR = 98%
- Nemotron-Mini-4B: 2 GB VRAM, no fine-tuning, 85.7% valid JSON — *preliminary*
- BRep JSON format successfully encodes branched midcurves for all models

Legacy, leaked, not re-measured. Train/test leakage on four base shapes; never reproduced under a clean split. Do not quote.

YHK

Phase II: Pros and Cons

Advantages

- Exact coordinates: no rasterisation loss
- BRep JSON handles branches natively
- Leverages vast LLM pretraining knowledge
- Easy to add new shapes to dataset

Limitations

- Coordinates are tokens: digit arithmetic is harder than regression
- Output validity not guaranteed (needs JSON repair pass)
- Inference latency: 2–10 s on GPU (vs $<$10 ms for Phase I)
- Nemotron results preliminary; QLoRA fine-tune ongoing

YHK

Phase III: Geometry-Based Approaches

Graph-to-Graph Dimension Reduction

YHK

Why Exact Geometry?

Limitations of the image approach motivate working directly on coordinates:

Image (Phase I)

- Rasterisation loses precision
- Output is a bitmap, not a polyline
- Branches not natively represented

Graph (Phase III)

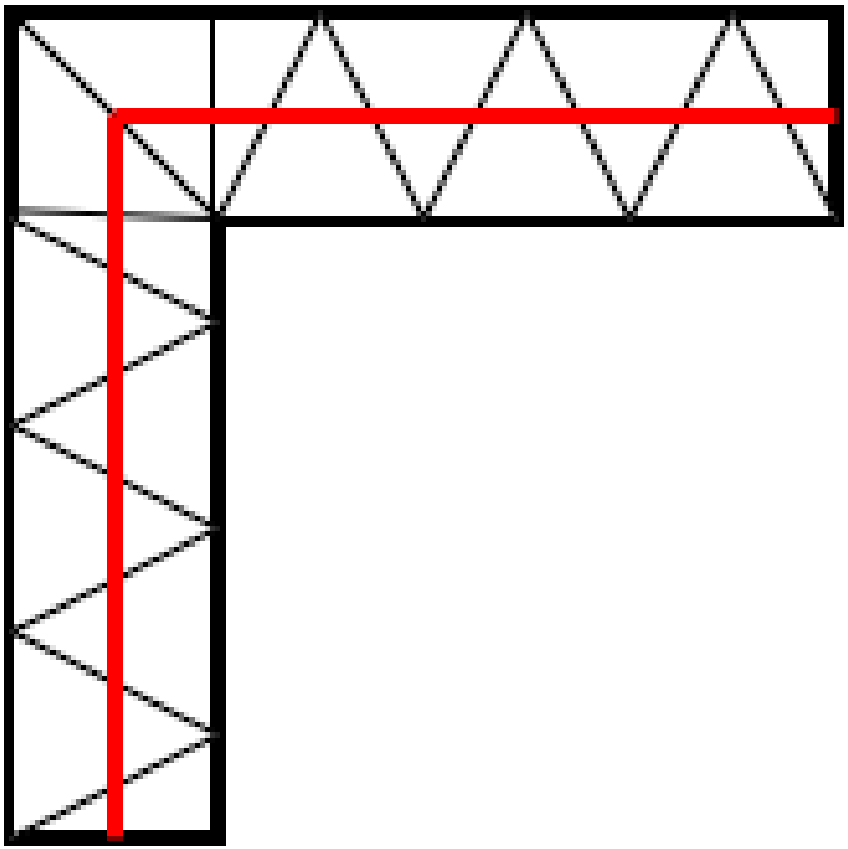
- Exact (x, y) node coordinates
- Output is a polyline graph
- Branched midcurves (T, Plus) native

Key insight: represent profile and midcurve as graphs, then learn the coarsening transformation.

YHK

Profile as a Graph

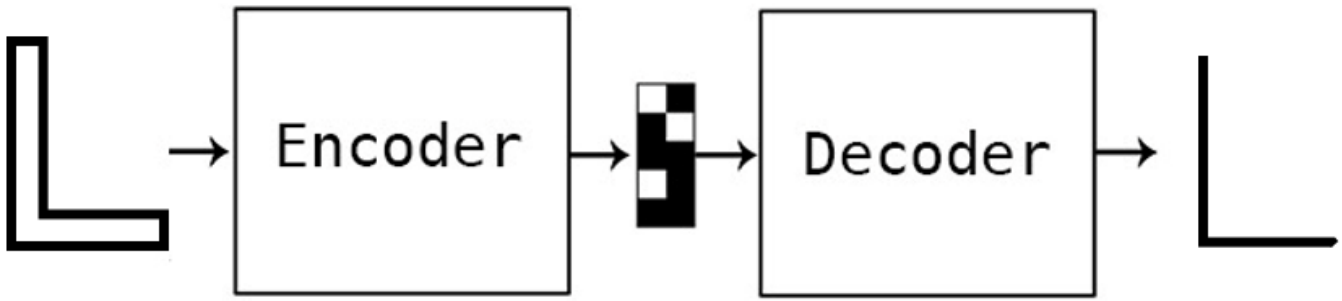
- **Nodes**: polygon corner points, features $= (x, y)$
- **Edges**: line segments connecting consecutive corners, feature $=$ length
- **Target**: midcurve graph with fewer nodes (coarsening)



Input shapes I, L, T, Plus: each becomes a closed ring graph.

YHK

Graph Transformer Architecture



- **Encoder**: TransformerConv layers with Laplacian Positional Encoding
- **Bottleneck**: TopK Pooling: keeps top-scored nodes
- **Decoder**: per-node MLP $\rightarrow$ coordinates; pairwise MLP $\rightarrow$ adjacency

YHK

Phase III: Pros and Cons

Advantages

- Exact geometry: no rasterisation
- Branches (T, Plus) handled natively
- Output is a clean graph, not a bitmap
- Invariant to point-listing order (with PE)
- Small data OK with geometric augmentation

Limitations

- Requires PyTorch Geometric: heavier stack
- TopK pooling fixes output node count
- Chamfer loss is not differentiable everywhere
- Limited training data (4 base shapes)

YHK

Geometric Deep Learning for the Midcurve

EquiCollapse: learning a collapse of the polygon's interior

Geometric Deep Learning in One Slide

- Bronstein, Bruna, Cohen and Veličković (2021) describe architectures by the same parts:
  - a **domain** (grid, graph, manifold)
  - a **symmetry group** acting on it
  - **equivariant layers**, local **coarsening**, and an invariant readout
- So the design questions become: what is the right domain, and which symmetries does the task really have?

GDL domain	Midcurve use
Grids	Image-based encoder-decoders
Graphs	Ring graph, attention over vertices
Groups	Rotation and reflection on coordinates
Geodesics	Distance through the interior, across opposite walls
Gauges	Local wall frames, boundary orientation

YHK

Which Symmetries Does the Midcurve Have?

Change to the polygon	Midcurve follows?
Different start vertex or listing direction	Yes
Translation, rotation, reflection: $E(2)$	Yes
Uniform scaling	Yes
Inserting a vertex on a straight wall	Yes
Relabelling output nodes	Yes
Stretching in one direction (affine)	**No**

- The medial axis is not affine-equivariant, so that symmetry must not be built in.
- The right domain is the polygon's **interior**: the informative neighbour of a wall is the opposite wall, far away along the boundary.

YHK

Starting Point: the Set Transformer

- 12 query slots, an existence flag and Hungarian matching predict the node set; edges come from an inside-polygon minimum spanning tree.
- Given true nodes, that tree recovers every true edge on these hole-free shapes. **Edges are not the bottleneck here**.
- It has no built-in rotation symmetry, and was trained with 90-degree turns only:

Test rotation	0°	5°	15°
Correct graph within $0.25w$	0.873	0.363	0.007

300 synthetic test shapes, published labels. Expected given the training; continuous rotation augmentation is the fair baseline.

YHK

A Bug Found Along the Way

- About 10% of generated samples have a spurious ground-truth node in the middle of a straight arm.
- Its position cannot be inferred from the profile, so no model can predict it.
- Cause: collinear merging compares raw lattice vectors, so it stops once spacings differ.

Correct graph within $0.25w$**, iid test**	**Published labels**	**Corrected**
Set transformer (100 epochs)	0.893	0.920
Classical Voronoi	0.813	0.862

The generator is fixed now; runs shown here were trained before the fix, with cleaned training labels, and report both label sets. The real shapes are unaffected.

YHK

Why a Plain Equivariant Decoder Fails

Claim. Suppose each query slot has a learned embedding and its position $f_q(P)$ is $E(2)$-equivariant. If a symmetry g maps P onto itself, then

$$f_q(P) = f_q(gP) = g\, f_q(P),$$

so every slot sits at a fixed point of the symmetry group of P, whatever the weights.

- I and Plus: the centre only
- T: the mirror axis only, so the bar's two ends are unreachable
- L: the diagonal only

Equivariant outputs keeping the input's symmetry is known in general (Smidt et al. 2021, Lawrence et al. 2025). Here, **every real benchmark shape is in the failure set**.

YHK

Slot Collapse: Numerical Check

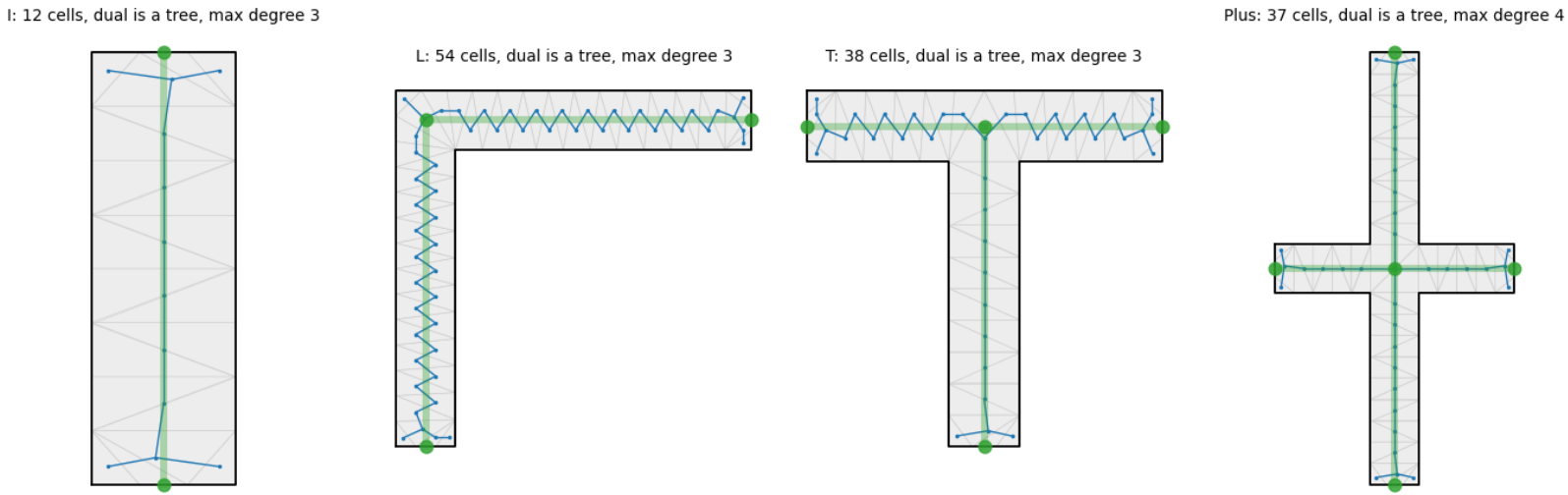
One small network overfitted to one shape; largest distance from a true node to its matched slot, in half-extent units (wall width is about 0.2 to 0.33):

Shape	Learned slots	Slots ended up	Wall-anchored slots
I	1.000	at the centre	0.003
L	1.382	on the diagonal	0.038
T	1.000	on the mirror axis	0.002
Plus	1.000	at the centre	0.006

Remedy: index outputs by input elements. A symmetry then permutes the slots rather than pinning them.

YHK

The Idea: Collapse the Interior



Grey: interior Delaunay cells. Blue: their dual graph. Green: true midcurve.

- With all samples on the boundary of a simple polygon, the dual graph is a tree.
- The midcurve looks like this tree with spurs removed and long chains contracted.
- So do not *generate* a graph: **learn which cells to keep** and contract the rest.

YHK

EquiCollapse: the Pipeline

1. **Canonicalise:** remove collinear vertices; estimate width $\hat{w} = 2A/P$.
2. **Build the complex:** sample walls at step $0.5\hat{w}$, Delaunay-triangulate, keep interior triangles, merge cocircular neighbours.
3. **Encode:** message passing on the dual graph with scalar and vector channels, using relative positions only.
4. **Predict per cell:** a key probability (invariant) and a position offset (equivariant).
5. **Decode, no learning:**
   - contract connected key cells into one output node
   - repeatedly remove non-key leaves (spurs)
   - remove non-key degree-2 cells

Loss: cell-wise BCE plus a robust offset loss. No matching, since cells give the correspondence.

YHK

What Holds by Construction, and What Does Not

By construction

- Node count is not limited by slots
- Output is a tree on hole-free shapes, for any weights (tested with random scores)
- Network is exactly $E(2)$-equivariant
- No slot collapse: outputs are anchored to cells

Not guaranteed or not tested

- The complex matching the polygon is checked per shape, not guaranteed
- The tree is abstract: drawn edges can still be geometrically wrong
- Holes: argued, but not implemented
- Width estimate on noisy boundaries

YHK

Other GDL Options Considered

Option	Verdict
Rotation augmentation (not GDL)	Cheap control; run first
EquiCollapse	Built
Equivariant decoder with query slots	Fails on symmetric shapes
Frame averaging as main method	Discontinuous at symmetric shapes
Steerable or tensor-field networks	More machinery than 2D needs
Graph diffusion	Models a distribution; this is a function
Autoregressive generation	Ordering is ambiguous on symmetric shapes
Cell networks, link prediction	Later, for holes

YHK

Closest Related Work

- **MidSurfNet** (Ye et al. 2026): learned face pairing on B-rep graphs for CAD mid-surfaces.
- **Mid-curve completion** (Boussuge and Marc 2021): a CNN patches problem regions of a medial axis.
- **Point2Skeleton** (Lin et al. 2021): unsupervised skeletal meshes from point clouds.
- **ConvPSNet** (Antony et al. 2025): learned skeleton of noisy 2D shapes; only a summary has been read so far.
- **EMPSN, CW Networks** (2021–2023): equivariant and cell-complex layers, on fixed-output tasks.
- **Chordal Axis Transform** (Prasad 1997): a rule-based skeleton on the same kind of structure.

Our searches found no prior work combining $E(2)$ equivariance, polygon input and learned contraction of the interior complex. They were not a systematic review, so this is not yet a novelty claim.

YHK

EquiCollapse: Rotation and Data Efficiency

Correct graph within $0.25w$	**Classical**	**Set transf. control**	**EquiCollapse**
iid, corrected labels	0.862	0.896	1.000
Rotated 15°	0.880	0.880	1.000
Rotated 45°	0.880	0.877	1.000
Degree-4 shapes, never trained on	0.841	0.798	1.000

iid, published labels	500 shapes	2000 shapes	8000 shapes
Set transformer control	0.299	0.559	0.810
EquiCollapse	0.901	0.904	0.904

1000 test shapes, one seed, trained before the generator fix. About 10% of published labels carry the spurious node, so 0.904 is the ceiling there. The control has the same cleaned training labels and continuous rotation augmentation.

YHK

EquiCollapse: Limits

Test-time jitter	$0.01w$	$0.02w$	$0.05w$	$0.1w$
Classical	0.293	0.200	0.073	0.003
Set transformer control	0.000	0.000	0.000	0.000
EquiCollapse	0.683	0.510	0.217	0.083

- Best of the three under noise, but it still degrades quickly; no model saw noise in training.
- Larger-than-training shapes (`ood_size`): 0.836 against 0.293 for the control.
- The data are hole-free rectilinear trees where the classical rule already scores 0.86. A perfect score here does **not** show generalisation to curved, thick or holed shapes: those are untested.
- Four real shapes only; one seed so far.

YHK

Phase I: Context

- Various applications need lower dimensional representation of shapes.
- Midcurve is one-dimensional (1D) representation of a two-dimensional (2D) planar shape.
- Used in animation, shape matching, retrieval, finite element analysis, etc.

YHK

Phase I: Approach

- Prior approaches: Thinning, Medial Axis Transform (MAT), Chordal Axis Transform (CAT), Straight Skeletons — all rule-based, not scalable.
- MidcurveNN: Encoder-Decoder neural network learns the mapping from 2D polygon image to 1D midcurve image in a supervised manner.
- Once trained, the network predicts midcurves for unseen polygon shapes.

YHK

Phase I: Results (Legacy)

Variant	Input	Loss	IoU	Branches
Simple AE	100×100	High	Low	$\times$
Dense AE	100×100	Medium	Med	$\times$
CNN AE	128×128	Low	High	$\times$

- CNN AE achieves $\approx 10\times$ lower loss than Dense AE
- All variants lose exact geometry (rasterisation trade-off)
- None can represent branched midcurves (T, Plus) natively

Legacy, leaked, not re-measured. Measured with train/test leakage on four base shapes; the $10\times$ figure has no current source. Do not quote.

YHK

Phase II: Motivation

- Traditional methods are predominantly rule-based: not scalable to arbitrary shapes.
- A novel Large Language Model approach attempts to build a generic, data-driven model.
- GPT-4 (few-shot) succeeds on simple shapes; open-source models struggle without fine-tuning — pointing to the value of domain-specific QLoRA fine-tuning.

YHK

Phase II: Findings (Legacy)

- BRep JSON representation solves the branching serialization problem for T- and Plus-shapes.
- QLoRA fine-tuning of Qwen2.5-7B achieves **PSR = 98%, MAE = 0.78** on the 4-shape benchmark.
- Nemotron-Mini-4B (2-shot, no fine-tuning): PSR = 85.7%, topology accuracy = 0.83, fits in ∼2 GB VRAM.
- Key challenge: coordinates are text tokens — digit arithmetic is harder for LLMs than neural regression.

Legacy, leaked, not re-measured. The PSR, MAE and topology figures come from the earlier runs with train/test leakage on four base shapes and were never reproduced under a clean split. Do not quote.

YHK

Phase II: MidcurveLLM Architecture

- Utilizes an Encoder-Decoder architecture and B-rep structures.
- Showcases promise, despite discrepancies with ground truths.

YHK

Phase II: Implications and Future Directions

- Points to the need for more extensive datasets and refined training parameters.
- Serves as a crucial catalyst for advancing midcurve computation methodologies.
- Invites further scrutiny and advancements in the transformative intersection of geometry and advanced machine learning.

YHK

Three Approaches: Summary

	Phase I Image	**Phase II Text/LLM**	**Phase III Geometry**
Input type	Bitmap (100–256px)	BRep JSON text	Graph (x, y)
Output type	Bitmap	BRep JSON text	Graph (x, y)
Exact coords	No	Yes	Yes
Branches	No	Yes (BRep)	Yes (native)
Legacy best*	CNN encoder-decoder	Qwen2.5-7B QLoRA / Nemotron-Mini-4B	Graph Transformer
Sub-variants	7 variants	Prompt, CodeT5, QLoRA, Nemotron3	GT + Graphormer FT
Key limitation	Rasterisation loss	Token arithmetic; repair needed	TopK fixes node count
Status	Fully impl.	Implemented	Implemented

*Legacy, leaked, not re-measured: these were the best of the earlier runs, not verified rankings.

Phase III here is the earlier Graph Transformer; its current status is in the geometry section.

YHK

Conclusions

- Midcurve computation is a **dimension-reduction problem**: 2D $\rightarrow$ 1D, graph-to-graph
- **Phase I** (image, legacy numbers, not re-measured): encoder-decoder with UNet achieves good results; rasterisation trade-off is the key limitation
- **Phase II** (LLM, legacy numbers, leaked, not re-measured): BRep JSON representation works; GPT-4 solves simple shapes via prompting; Nemotron-Mini-4B achieves 85.7% valid JSON + 0.83 topology accuracy base-model; QLoRA fine-tuning (Qwen, Gemma, Nemotron) shows promise but needs more data
- **Phase III** (geometry): Graph Transformer operates directly on polygon coordinates, handles branches natively, and outputs a clean polyline graph without post-processing

YHK

Next Frontier: Graph-to-Graph Translation

- The ultimate formulation is **graph-to-graph**: profile graph $\Rightarrow$ midcurve graph
- This is a **graph coarsening / dimension-reduction** problem, currently underexplored in geometric deep learning
- Promising directions:
  - Non-auto-regressive Graph Transformers (Phase III, this work)
  - Diffusion-based graph generation conditioned on input topology
  - Geometric Graph Normalizing Flows
  - Integration of LLM structural reasoning (Phase II) with geometric constraints (Phase III)
- Goal: a model that generalises to **any 2D thin polygon** (not just I, L, T, Plus) and produces an exact, branch-aware midcurve without rasterisation or post-processing

Graph-to-Graph is the next frontier for MidcurveNN.

YHK

References

- Kulkarni, Y. H.; Deshpande, S. Medial Object Extraction - A State of the Art In International Conference on Advances in Mechanical Engineering, SVNIT, Surat, 2010.
- Kulkarni, Y. H.; Sahasrabudhe, A.D.; Kale, M.S Dimension-reduction technique for polygons In International Journal of Computer Aided Engineering and Technology, Vol. 9, No. 1, 2017.
- Kulkarni, Y.H. "MidcurveNN: Encoder-Decoder Neural Network for Computing Midcurve of a Thin Polygon", CAD Conference 2021, Barcelona, pp. 223–225.
- Kulkarni, Y.H. "Computing Midcurve with Multi-Layer and Convolutional Neural Networks", CAD & Applications, Vol. 19(6), 2022, pp. 1154–1161.
- Chollet, F. Building Autoencoders in Keras In https://blog.keras.io/building-autoencoders-in-keras.html , 2019.
- Video: https://www.youtube.com/embed/ZY0nuykqgoE?featureōembed
- Presentation: https://drive.google.com/file/d/1Tx5JJK1_LUflMTW-B43HNN2GDMKJMOxR/preview
- Short paper: https://vixra.org/abs/1904.0429
- Github repo, source code: https://github.com/yogeshhk/MidcurveNN
- Kulkarni, Y.H. "Midcurve Computation using Large Language Models", ICONIEA 2024, IIT Kharagpur (extended abstract).
- Kulkarni, Y.H. "LLM Based Midcurve Generation", ICCCIT 2025, IEEE Xplore.

YHK

References: Geometric Deep Learning

- Bronstein, M.; Bruna, J.; Cohen, T.; Veličković, P. Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges. arXiv:2104.13478, 2021.
- Satorras, V. G.; Hoogeboom, E.; Welling, M. E(n) Equivariant Graph Neural Networks. ICML 2021.
- Smidt, T.; Geiger, M.; Miller, B. Finding Symmetry Breaking Order Parameters with Euclidean Neural Networks. Phys. Rev. Research, 2021.
- Lawrence, H.; Portilheiro, V.; Zhang, Y.; Kaba, S.-O. Improving Equivariant Networks with Probabilistic Symmetry Breaking. ICLR 2025.
- Eijkelboom, F.; Hesselink, R.; Bekkers, E. E(n) Equivariant Message Passing Simplicial Networks. ICML 2023.
- Bodnar, C. et al. Weisfeiler and Lehman Go Cellular: CW Networks. NeurIPS 2021.
- Prasad, L. Morphological Analysis of Shapes. 1997.
- Ye, L. et al. MidSurfNet: Learning Face Pairing for Mid-surface Abstraction of Thin-walled CAD Models. arXiv:2606.01891, 2026.
- Boussuge, F.; Marc, R. Mid-Curve Completion Using Convolutional Neural Network. International Meshing Roundtable, 2021.
- Lin, C. et al. Point2Skeleton: Learning Skeletal Representations from Point Clouds. CVPR 2021.
- Antony et al. ConvPSNet. SIGGRAPH Asia 2025 Technical Communications.

YHK

Thanks ...

- Search **"Yogesh Haribhau Kulkarni"** on Google and follow me on LinkedIn and Medium
- Office Hours: Saturdays, 2 to 5pm (IST); Free-Open to all; email for appointment.
- Email: yogeshkulkarni at yahoo dot com

(https://www.linkedin.com/in/yogeshkulkarni/, QR by Hugging Face QR-code-AI-art-generator, with prompt as "Follow me")



YHK